

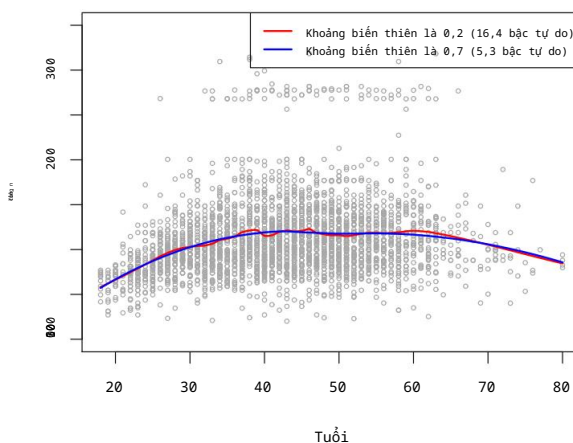
Thuật toán 7.1 Hồi quy cục bộ tại $X = x_0$

1. Tập hợp phân số $s = k/n$ các điểm huấn luyện có xi gần nhất đến x_0 .
2. Gán trọng số $K_{i0} = K(x_i, x_0)$ cho mỗi điểm trong vùng lân cận này. sao cho điểm xa nhất so với x_0 có trọng số bằng 0, và điểm gần nhất có trọng số bằng 0. có trọng lượng cao nhất. Tất cả các láng giềng gần nhất khác, ngoại trừ k láng giềng gần nhất này, đều có trọng lượng số không.
3. Thực hiện hồi quy bình phương tối thiểu có trọng số của yi trên xi bằng cách sử dụng các trọng số đã đề cập ở trên, bằng cách tìm $\hat{\beta}_0$ và $\hat{\beta}_1$ sao cho tối thiểu hóa

$$\sum_{i=1}^N K_{i0} (y_i - \beta_0 - \beta_1 x_i)^2. \quad (7.14)$$

4. Giá trị phù hợp tại x_0 được cho bởi $\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0$.

Hồi quy tuyến tính cục bộ



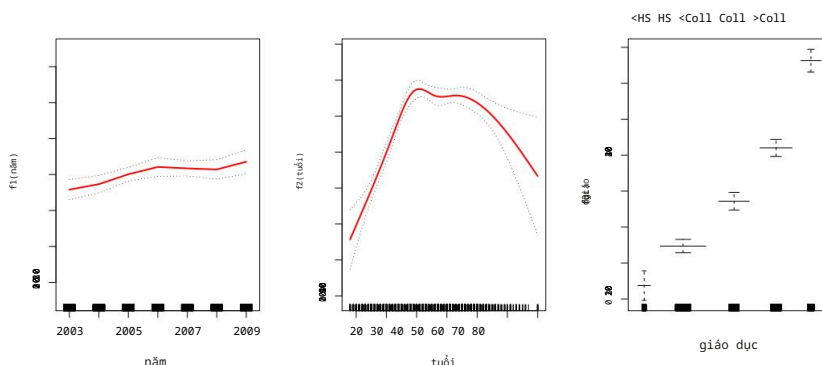
HÌNH 7.10. Đường thẳng phù hợp cục bộ với dữ liệu Tiền lương. Khoảng cách xác định phân số của dữ liệu được sử dụng để tính toán độ phù hợp tại mỗi điểm mục tiêu.

7.7 Mô hình cộng tính tổng quát

Trong các phần 7.1-7.6, chúng tôi trình bày một số phương pháp dự đoán linh hoạt phản hồi Y dựa trên một biến dự báo X duy nhất. Các phương pháp này có thể có thể được xem như là sự mở rộng của hồi quy tuyến tính đơn giản. Ở đây, chúng ta khám phá vấn đề dự đoán linh hoạt Y dựa trên một số biến dự báo, $X_1, \dots, X_p$. Điều này tương đương với việc mở rộng phương pháp hồi quy tuyến tính bội.

Mô hình cộng tính tổng quát (GAM) cung cấp một khuôn khổ tổng quát cho các mô hình tổng quát. Mở rộng mô hình tuyến tính tiêu chuẩn bằng cách cho phép các hàm phi tuyến tính của mỗi phần tử của các biến, đồng thời duy trì tính chất cộng tính. Giống như các mô hình tuyến tính, GAMs có thể áp dụng cho cả phản hồi định lượng và định tính. Trước tiên, chúng ta xét tính cộng gộp.

chất phụ gia
người mẫu



HÌNH 7.11. Đối với dữ liệu về tiền lương, biểu đồ thể hiện mối quan hệ giữa từng đặc điểm và phản hồi, tiền lương, trong mô hình phù hợp (7.16). Mỗi biểu đồ hiển thị mô hình phù hợp hàm và sai số chuẩn từng điểm. Hai hàm đầu tiên là các hàm spline tự nhiên về năm và tuổi, với bốn và năm bậc tự do tương ứng. Thứ ba Hàm số này là một hàm bậc thang, phù hợp với biến định tính về giáo dục.

Kiểm tra GAM để tìm câu trả lời định lượng trong Phần 7.7.1, và sau đó để tìm một Phản hồi định tính trong Mục 7.7.2.

7.7.1 Mô hình GAM cho các bài toán hồi quy

Một cách tự nhiên để mở rộng mô hình hồi quy tuyến tính bội.

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} + \epsilon_i$$

để cho phép các mối quan hệ phi tuyến tính giữa mỗi đặc điểm và

Giải pháp là thay thế mỗi thành phần tuyến tính $\beta_j x_{ij}$ bằng một hàm phi tuyến (mượt mà) $f_j(x_{ij})$. Khi đó, chúng ta sẽ viết mô hình như sau:

$$y_i = \beta_0 + \sum_{j=1}^p f_j(x_{ij}) + \epsilon_i$$

$$= \beta_0 + f_1(x_{i1}) + f_2(x_{i2}) + \dots + f_p(x_{ip}) + \epsilon_i. \quad (7.15)$$

Đây là một ví dụ về GAM. Nó được gọi là mô hình cộng tính vì chúng ta

Tính một giá trị f_j riêng biệt cho mỗi X_j , sau đó cộng tất cả các giá trị đó lại với nhau. đóng góp.

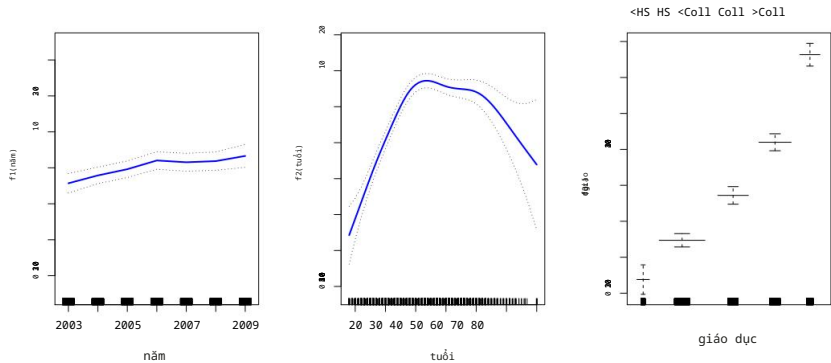
Trong các phần 7.1-7.6, chúng ta sẽ thảo luận về nhiều phương pháp để khớp các hàm với một biến đơn. Cái hay của GAM là chúng ta có thể sử dụng các phương pháp này như những khối xây dựng để phù hợp với mô hình cộng tính. Trên thực tế, đối với hầu hết các trường hợp... Với những phương pháp mà chúng ta đã thấy trong chương này, điều này có thể được thực hiện một cách khá dễ dàng. một cách đơn giản. Ví dụ, hãy xem xét các đường cong spline tự nhiên và nhiệm vụ khớp chúng lại với nhau. mô hình

$$\text{lương} = \beta_0 + f_1(\text{năm}) + f_2(\text{tuổi}) + f_3(\text{trình độ học vấn}) + \dots \quad (7.16)$$

về dữ liệu tiền lương. Ở đây, năm và tuổi là các biến định lượng, trong khi...

Giáo dục có tính chất biến đổi với năm cấp độ: <HS, HS, <College, College, >College.

Đề cập đến số năm học trung học hoặc đại học mà một cá nhân đã hoàn thành. Chúng tôi sử dụng spline tự nhiên để khớp hai hàm đầu tiên. Chúng tôi



HÌNH 7.12. Chi tiết tương tự như Hình 7.11, nhưng giờ đây f1 và f2 đang được làm mịn. Các hàm spline có bốn và năm bậc tự do tương ứng.

Điều chỉnh hàm thứ ba bằng cách sử dụng một hằng số riêng cho mỗi cấp độ, thông qua phương pháp thông thường. Phương pháp biến giả được trình bày trong Mục 3.3.1.

Hình 7.11 hiển thị kết quả phù hợp với mô hình (7.16) bằng cách sử dụng ít nhất hình vuông. Điều này rất dễ thực hiện, vì như đã thảo luận trong Phần 7.4, các đường cong spline tự nhiên có thể được xây dựng bằng cách sử dụng một tập hợp các hàm cơ sở được lựa chọn phù hợp. Do đó, toàn bộ mô hình chỉ là một phép hồi quy lớn dựa trên các biến cơ sở spline. và các biến giả, tất cả được gói gọn trong một ma trận hồi quy lớn.

Hình 7.11 rất dễ hiểu. Bảng bên trái cho thấy rằng Nếu giữ nguyên **tuổi tác** và **trình độ học vấn**, **mức lương** có xu hướng tăng nhẹ theo từng **năm**; Điều này có thể là do lạm phát. Bảng ở giữa cho thấy rằng việc nắm giữ Nếu **trình độ học vấn** và **tuổi tác** cố định, **mức lương** thường cao nhất ở độ **tuổi** trung bình và thấp nhất ở người rất trẻ và rất già. (Phần cuối cùng không liên quan đến đoạn văn, nên cần được dịch chính xác hơn.) Biểu đồ cho thấy rằng, khi giữ nguyên **tuổi** và **năm**, **mức lương** có xu hướng tăng lên. Về **trình độ học vấn**: người càng có trình độ học vấn cao thì lương càng cao. trung bình. Tất cả những phát hiện này đều dễ hiểu.

Hình 7.12 hiển thị một bộ ba đồ thị tương tự, nhưng lần này f1 và f2 là Các spline làm mịn với bốn và năm bậc tự do tương ứng. Việc khớp một GAM với spline làm mịn không đơn giản như việc khớp một GAM thông thường.

với đường cong spline tự nhiên, vì trong trường hợp làm mịn spline, phương pháp bình phương tối thiểu được sử dụng. Không thể sử dụng được. Tuy nhiên, các phần mềm tiêu chuẩn như gói **Python** vẫn có thể được sử dụng. **Pygam** có thể được sử dụng để hiệu chỉnh các mô hình GAM bằng cách sử dụng spline làm mịn, thông qua một phương pháp được gọi là hiệu chỉnh ngược (backfitting). Phương pháp này hiệu chỉnh một mô hình bao gồm nhiều biến dự đoán bằng cách liên tục cập nhật độ phù hợp cho từng biến dự đoán một, giữ nguyên giá trị ban đầu. những thứ khác đã được sửa. Ưu điểm của phương pháp này là mỗi khi chúng ta cập nhật một hàm, chúng ta chỉ cần áp dụng phương pháp khớp cho biến đó vào một phần. dư.

Các hàm được khớp trong Hình 7.11 và 7.12 trông khá giống nhau. Trong hầu hết các trường hợp, các tình huống, sự khác biệt trong các GAM thu được bằng cách sử dụng spline làm mịn

Sau với các đường cong spline tự nhiên thì kích thước nhỏ hơn.

Ví dụ, phần dư riêng phần 6A cho X3 có dạng $y_i = f_1(x_{i1}) + f_2(x_{i2})$. Nếu chúng ta Biết f1 và f2, sau đó ta có thể ước lượng f3 bằng cách coi phần dư này như một phần hồi trong một mô hình phi tuyến tính. Hồi quy trên X3.

Chúng ta không nhất thiết phải sử dụng hàm spline làm khối xây dựng cho GAM: chúng ta hoàn toàn có thể sử dụng hồi quy cục bộ, hồi quy đa thức, hoặc bất kỳ sự kết hợp nào của các phương pháp đã thấy trước đó trong chương này để tạo ra một GAM. GAM sẽ được nghiên cứu chi tiết hơn trong phòng thí nghiệm ở cuối chương này.

Ưu điểm và nhược điểm của GAM

Trước khi tiếp tục, chúng ta hãy tóm tắt những ưu điểm và hạn chế của GAM.

Mô hình GAM cho phép chúng ta áp dụng một hàm fj phi tuyến tính để chúng ta có thể cho mỗi biến Xj, tự động mô hình hóa các mối quan hệ phi tuyến tính mà hồi quy tuyến tính tiêu chuẩn sẽ bỏ sót. Điều này có nghĩa là chúng ta không cần phải tự mình thử nghiệm nhiều phép biến đổi khác nhau trên từng biến riêng lẻ.

Các phép khớp phi tuyến tính có khả năng đưa ra dự đoán chính xác hơn cho phản hồi Y.

Vì mô hình này là mô hình cộng, chúng ta có thể xem xét tác động của từng biến Xj lên Y một cách riêng lẻ trong khi giữ cố định tất cả các biến khác.

Độ trơn tru của hàm fj đối với biến Xj có thể được tóm tắt thông qua bậc tự do.

- Hạn chế chính của GAM là mô hình chỉ có thể được xem là cộng tính. Với nhiều biến số, các tương tác quan trọng có thể bị bỏ sót. Tuy nhiên, cũng giống như hồi quy tuyến tính, chúng ta có thể thêm thủ công các thuật ngữ tương tác vào mô hình GAM bằng cách bao gồm các biến dự đoán bổ sung có dạng Xj × Xk. Ngoài ra, chúng ta có thể thêm các hàm tương tác chiều thấp có dạng fjk(Xj, Xk) vào mô hình; các thuật ngữ như vậy có thể được điều chỉnh bằng cách sử dụng các bộ làm mịn hai chiều như hồi quy cục bộ, hoặc các hàm spline hai chiều (không được đề cập ở đây).

Đối với các mô hình tổng quát hoàn toàn, chúng ta phải tìm kiếm các phương pháp linh hoạt hơn nữa như rừng ngẫu nhiên và boosting, được mô tả trong Chương 8. GAM cung cấp một sự thỏa hiệp hữu ích giữa các mô hình tuyến tính và các mô hình phi tham số hoàn toàn.

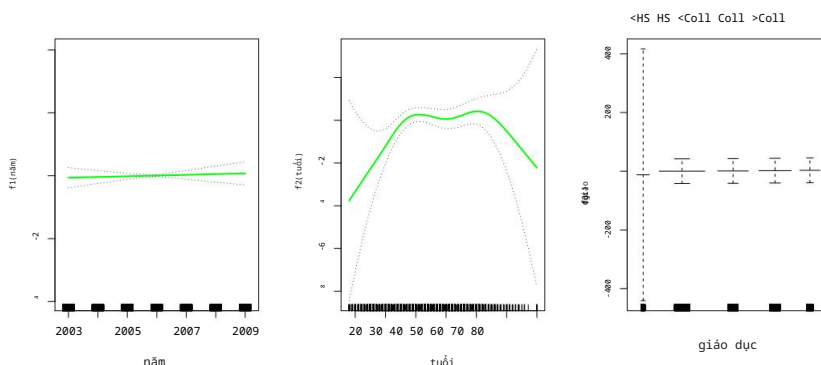
7.7.2 Mô hình GAM cho các bài toán phân loại

GAM cũng có thể được sử dụng trong các tình huống mà Y là biến định tính. Để đơn giản, ở đây chúng ta giả sử Y nhận các giá trị 0 hoặc 1, và gọi $p(X) = \Pr(Y=1|X)$ là xác suất có điều kiện (cho trước các biến dự đoán) rằng phản hồi bằng một. Nhớ lại mô hình hồi quy logistic (4.6):

$$\text{nhật ký} \quad \frac{p(X)}{1 - p(X)} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p. \quad (7.17)$$

Vế bên trái là logarit của tỷ lệ cược $P(Y=1|X)$ so với $P(Y=0|X)$, được biểu diễn như một hàm tuyến tính của các biến dự đoán (7.17). Một cách tự nhiên để mở rộng (7.17) để cho phép các mối quan hệ phi tuyến tính là sử dụng mô hình

$$\text{nhật ký} \quad \frac{p(X)}{1 - p(X)} = \beta_0 + f_1(X_1) + f_2(X_2) + \dots + f_p(X_p). \quad (7.18)$$



HÌNH 7.13. Đối với dữ liệu *Tiền lương*, mô hình GAM hồi quy logistic được đưa ra trong (7.19) được khớp với phân hồi nhị phân $I(\text{lương} > 250)$. Mỗi đồ thị hiển thị hàm được khớp. và sai số chuẩn từng điểm. Hàm đầu tiên là tuyến tính theo *năm*, hàm thứ hai là tuyến tính theo năm. hàm spline làm mịn với năm bậc tự do về *tuổi tác*, và bậc tự do thứ ba là hàm bậc thang cho *giáo dục*. Có sai số chuẩn rất lớn đối với phần đầu tiên. trình độ *<Trung học phổ thông*.

Phương trình 7.18 là một mô hình GAM hồi quy logistic. Nó có tất cả các ưu điểm và tính chất tương tự như đã thảo luận ở phần trước đối với các phân hồi định lượng.

Chúng tôi sử dụng mô hình GAM để phân tích dữ liệu về *tiền lương* nhằm dự đoán xác suất rằng...

Thu nhập của một cá nhân vượt quá 250.000 đô la mỗi năm. Mô hình GAM mà chúng tôi áp dụng có hình thức

$$\text{nhật ký} \frac{p(X)}{1 - p(X)} = \beta_0 + \beta_1 \times \text{năm} + f_2(\text{tuổi}) + f_3(\text{giáo dục}), \quad (7.19)$$

Ở đây

$$p(X) = \text{Pr}(\text{lương} > 250 | \text{năm}, \text{tuổi}, \text{trình độ học vấn}).$$

Một lần nữa, f_2 được điều chỉnh bằng cách sử dụng đường cong spline làm mịn với năm bậc tự do. và f_3 được điều chỉnh như một hàm bậc thang, bằng cách tạo ra các biến giả cho mỗi phần tử. trình độ giáo dục. Kết quả phù hợp được thể hiện trong Hình 7.13. Bảng cuối cùng Trông có vẻ đáng ngờ, với khoảng tin cậy rất rộng cho mức *<HS*. Trên thực tế, không có giá trị phản hồi nào bằng một cho danh mục đó: không có cá nhân nào có giá trị nhỏ hơn Chỉ cần tốt nghiệp trung học phổ thông là có thể kiếm được hơn 250.000 đô la mỗi năm. Vì vậy, chúng tôi sẽ trang bị lại. GAM, không bao gồm những cá nhân có trình độ học vấn dưới trung học phổ thông. Mô hình thu được được thể hiện trong Hình 7.14. Tương tự như trong Hình 7.11 và 7.12, Cả ba tấm đều có tỷ lệ chiều dọc tương tự nhau. Điều này cho phép chúng ta đánh giá bằng mắt thường. sự đóng góp tương đối của từng biến số. Chúng ta nhận thấy rằng *tuổi tác* và *Trình độ học vấn* có ảnh hưởng lớn hơn nhiều so với *số năm học* đến xác suất được Người có thu nhập cao.

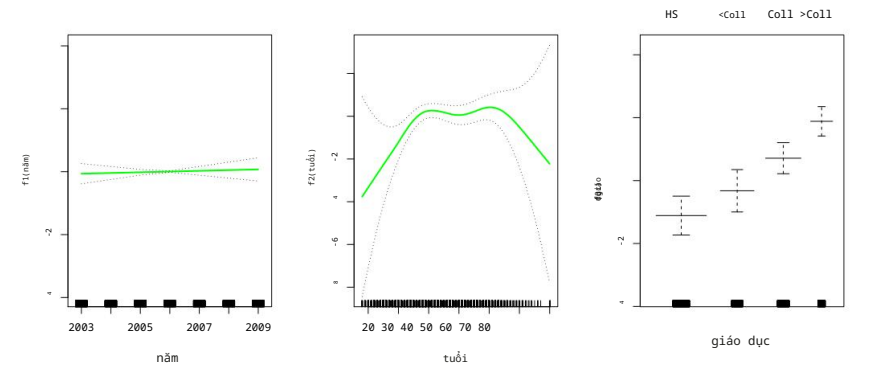
7.8 Bài thực hành: Mô hình hóa phi tuyến tính

Trong bài thực hành này, chúng tôi sẽ trình bày một số mô hình phi tuyến đã được thảo luận trong...

Chương này. Chúng ta sử dụng dữ liệu về *tiền lương* làm ví dụ minh họa và chỉ ra rằng

Nhiều quy trình khớp phi tuyến phức tạp được thảo luận có thể dễ dàng được

Được triển khai bằng *Python*.



HÌNH 7.14. Mô hình tương tự được điều chỉnh như trong Hình 7.13, lần này loại trừ... các quan sát cho thấy trình độ giáo dục <HS. Giờ đây, chúng ta thấy rằng trình độ giáo dục tăng lên. Thường gắn liền với mức lương cao hơn.

Như thường lệ, chúng ta bắt đầu với một số mặt hàng nhập khẩu quen thuộc.

```
Trong [1]: import numpy as np, pandas as pd
from matplotlib.pyplot import subplots
import statsmodels.api as sm
from ISLP import load_data
from ISLP.models import (summarize,
                          đa,
                          ModelSpec dưới dạng MS)
from statsmodels.stats.anova import anova_lm
```

Chúng tôi lại tiếp tục thu thập các vật tư nhập khẩu mới cần thiết cho phòng thí nghiệm này. Nhiều trong số đó là... Được phát triển đặc biệt cho gói ISLP .

```
Trong [2]: từ pygam import (s as s_gam,
                          l as l_gam,
                          f as f_gam,
                          LinearGAM,
                          LogisticGAM)

from ISLP.transforms import (BSpline,
                              Đường cong tự nhiên)

from ISLP.models import bs, ns
from ISLP.pygam import (approx_lam,
                        bậc tự do,
                        vẽ dưới dạng plot_gam,
                        anova as anova_gam)
```

7.8.1 Hồi quy đa thức và hàm bậc thang

Chúng ta bắt đầu bằng cách trình bày cách tái tạo Hình 7.1 . Hãy bắt đầu nào. bằng cách tải dữ liệu.

```
Trong [3]: Wage = load_data('Wage')
y = Wage['wage']
tuổi = Lương['tuổi']
```

Trong hầu hết bài thực hành này, câu trả lời của chúng ta là `Wage['wage']`, mà chúng ta đã lưu trữ dưới dạng `y` ở trên. Như trong Phần 3.6.6, chúng ta sẽ sử dụng hàm `poly()` để tạo ma trận mô hình phù hợp với đa thức bậc 4 theo `tuổi`.

```
Trong [4]: poly_age = MS([poly('age', degree=4)]).fit(Wage)
M = sm.OLS(y, poly_age.transform(Wage)).fit() summarize(M)
```

Ra ngoài[4]:

	điểm giao	hệ số lỗi chuẩn	t	P> t
	111,7036	0,729	153,283	0,000
poly(tuổi, bậc=4)[0]	447.0679	39.915	11.201	0.000
poly(tuổi, bậc=4)[1]	-478.3158	39.915	-11.983	0.000
poly(tuổi, bậc=4)[2]	125.5217	39.915	poly(tuổi, bậc=4)[3]	-77.9112
	-1.952	0.051		3,145
				0,002

Đa thức này được xây dựng bằng cách sử dụng hàm `poly()`, hàm này tạo ra một bộ chuyển đổi đặc biệt `Poly()` (sử dụng thuật ngữ của `sklearn` cho các phép biến đổi đặc trưng như `PCA()` được thấy trong Phần 6.5.3) cho phép dễ dàng đánh giá đa thức tại các điểm dữ liệu mới. Ở đây, `poly()` được gọi là hàm hỗ trợ và thiết lập phép biến đổi; `Poly()` là hàm chính thực hiện việc tính toán phép biến đổi. Xem thêm phần thảo luận về các phép biến đổi ở trang 118.

máy biến áp

người trợ giúp

Trong đoạn mã trên, dòng đầu tiên thực thi phương thức `'fit()'` sử dụng `dataframe 'Wage'`. Thao tác này tính toán lại và lưu trữ dưới dạng thuộc tính bất kỳ tham số nào cần thiết cho phương thức `'Poly()'` trên dữ liệu huấn luyện, và các tham số này sẽ được sử dụng trong tất cả các lần đánh giá tiếp theo của phương thức `'transform()'`. Ví dụ, nó được sử dụng ở dòng thứ hai, cũng như trong hàm vẽ đồ thị được phát triển bên dưới.

Giờ chúng ta sẽ tạo một lưới các giá trị cho `độ tuổi` mà chúng ta muốn đưa ra dự đoán.

```
Trong [5]: age_grid = np.linspace(age.min(), age.max(), 100)

age_df = pd.DataFrame({'age': age_grid})
```

Cuối cùng, chúng ta muốn vẽ đồ thị dữ liệu và thêm đường cong phù hợp từ đa thức bậc bốn. Vì chúng ta sẽ tạo một số đồ thị tương tự bên dưới, trước tiên chúng ta viết một hàm để tạo tất cả các thành phần và tạo ra đồ thị. Hàm của chúng ta nhận vào một thông số kỹ thuật của mô hình (ở đây là một cơ sở được xác định bởi một phép biến đổi), cũng như một lưới các giá trị `tuổi`. Hàm này tạo ra một đường cong phù hợp cũng như các dải tin cậy 95%. Bằng cách sử dụng một đối số cho `cơ sở`, chúng ta có thể tạo và vẽ đồ thị các kết quả với một số phép biến đổi khác nhau, chẳng hạn như các đường cong `spline` mà chúng ta sẽ thấy ngay sau đây.

```
Trong [6]: def plot_wage_fit(age_df,
                             cơ sở,
                             tiêu đề):

    X = basis.transform(Wage)
    Xnew = basis.transform(age_df)
    M = sm.OLS(y, X).fit() preds =
    M.get_prediction(Xnew) bands =
    preds.conf_int(alpha=0.05) fig, ax =
    subplots(figsize=(8,8)) ax.scatter(age,
                                         y,
```

```
        màu_mặt='xám',
        alpha=0,5)

for val, ls in zip([preds.predicted_mean ,
                  các_dải[:,0],
                  các_dải[:,1]],
                  ['b','r--','r--']):
    ax.plot(age_df.values, val, ls, linewidth=3)
ax.set_title(title, fontsize=20)
ax.set_xlabel('Tuổi', fontsize=20)
ax.set_ylabel('Lương', fontsize=20);
trả_lại ax
```

Chúng tôi thêm tham số `alpha` vào `ax.scatter()` để tăng độ trong suốt.
đến các điểm. Điều này cung cấp một chỉ báo trực quan về mật độ. Hãy chú ý đến cách sử dụng của hàm `zip()` trong vòng lặp `for` ở trên (xem Mục 2.3.8). Chúng ta có Ba đường thẳng cần vẽ, mỗi đường có màu sắc và kiểu đường khác nhau. Tại đây `zip()` Nó tiện lợi nhóm các phần tử này lại với nhau thành các trình lặp trong vòng lặp.
Bây giờ chúng ta sẽ vẽ đồ thị biểu diễn sự phù hợp của đa thức bậc bốn bằng cách sử dụng hàm này.

trình lặp

```
Trong [7]: plot_wage_fit(age_df,
                        đa_tuổi,
                        'Đa thức bậc 4');
```

Với hồi quy đa thức, chúng ta phải quyết định bậc của đa thức cần sử dụng. Đôi khi chúng ta chỉ cần quyết định đại khái và dùng đa thức bậc hai hoặc bậc ba.
các đa thức bậc cao, đơn giản chỉ để có được sự phù hợp phi tuyến tính. Nhưng chúng ta có thể tạo ra như vậy đưa ra quyết định một cách có hệ thống hơn. Một cách để làm điều này là thông qua các bài kiểm tra giả thuyết, mà chúng tôi sẽ trình bày ở đây. Bây giờ chúng tôi sẽ xây dựng một loạt các mô hình, bao gồm... từ đa thức bậc nhất đến đa thức bậc năm, và tìm cách xác định mô hình đơn giản nhất đủ để giải thích mối quan hệ giữa tiền lương và tuổi tác. Chúng tôi sử dụng hàm `anova_lm()` , thực hiện một loạt các phép tính. Kiểm định ANOVA. Phân tích phương sai (ANOVA) kiểm định giả thuyết không rằng mô hình M1 đủ để giải thích dữ liệu, trái ngược với giả thuyết thay thế.
giả thuyết cho rằng cần một mô hình M2 phức tạp hơn. Sự xác định Dựa trên phép thử F. Để thực hiện phép thử, các mô hình M1 và M2 phải đáp ứng các điều kiện sau: phải lồng nhau: không gian được tạo bởi các biến dự đoán trong M1 phải là một không gian con. của không gian được tạo ra bởi các biến dự đoán trong M2. Trong trường hợp này, chúng tôi đã xây dựng năm mô hình đa thức khác nhau và lần lượt so sánh mô hình đơn giản hơn với Mô hình phức tạp hơn.

phân tích của phương sai

```
Trong [8]: models = [MS([poly('age', degree=d)])
                    đối_với_d_trong_phạm_vi(1, 6)]
Xs = [model.fit_transform(Wage) for model in models]
anova_lm(*[sm.OLS(y, X_).fit()
           đối_với_X_ trong Xs])
```

Ra ngoài[8]:	df_resid	ssr	df_diff	0.0	ss_diff	F	Pr(>F)
0	2998.0	5.022e+06			NaN	NaN	NaN
1	2997.0	4.793e+06	1.0	228786.010	143.593	2.364e-32	
2	2996.0	4.778e+06	1.0	15755.694		9,889	1,679e-03
3	2995.0	4.772e+06	1.0	6070.152		3.810	5.105e-02

7. Theo ngôn ngữ Python , "iterator" là một đối tượng có số lượng giá trị hữu hạn, có thể

có thể được lặp đi lặp lại, như trong một vòng lặp.

Thay vì sử dụng kiểm định giả thuyết và ANOVA, chúng ta có thể lựa chọn... bậc đa thức được xác định bằng phương pháp kiểm định chéo, như đã thảo luận trong Chương 5. Tiếp theo, chúng ta xem xét nhiệm vụ dự đoán liệu một cá nhân có kiếm được hơn 250.000 đô la mỗi năm hay không. Chúng ta tiến hành tương tự như trước, ngoại trừ việc trước tiên chúng ta tạo ra vectơ phản hồi thích hợp, và sau đó áp dụng hàm `glm()` sử dụng họ phân phối nhị thức để phù hợp với mô hình hồi quy logistic đa thức.

```
Trong [12]: X = poly_age.transform(Wage) high_earn
= Wage['high_earn'] = y > 250 # viết tắt glm = sm.GLM(y > 250,

X, family=sm.families.Binomial())

B = glm.fit()
summarize(B)
```

Ra ngoài[12]:

	hệ số lỗi chuẩn	z	P> z
hệ số chặn	-4.3012 0.345 -12.457 0.000	poly(tuổi, bậc=4)[0]	71.9642 26.133 2.754
0.006	poly(tuổi, bậc=4)[1] -85.7729 35.929 -2.387 0.017	poly(tuổi, bậc=4)[2]	34.1626 19.697 1.734 0.083
poly(tuổi, bậc=4)[3]	-47.4008 24.105 -1.966 0.049		

Một lần nữa, chúng ta thực hiện dự đoán bằng phương thức `get_prediction()` .

```
Trong [13]: newX = poly_age.transform(age_df) preds =
B.get_prediction(newX) bands =
preds.conf_int(alpha=0.05)
```

Bây giờ chúng ta sẽ vẽ đồ thị biểu diễn mối quan hệ ước tính.

```
Trong [14]: fig, ax = subplots(figsize=(8,8)) rng =
np.random.default_rng(0) ax.scatter(age +
0.2 *
rng.uniform(size=y.shape[0]), np.where(high_earn, 0.198,
0.002), fc='gray', marker='|')

for val, ls in zip([preds.predicted_mean , bands[:,0],
bands[:,1]],
['b', 'r--', 'r--']):

ax.plot(age_df.values, val, ls, linewidth=3) ax.set_title('Đa
thức bậc 4 ', fontsize=20) ax.set_xlabel('Tuổi', fontsize=20)
ax.set_ylim([0,0.2]) ax.set_ylabel('P(Lương >
250)', fontsize=20);
```

Chúng tôi đã vẽ các giá trị `tuổi` tương ứng với các quan sát có giá trị `tiền lương` trên 250 dưới dạng các điểm màu xám ở phía trên biểu đồ, và những giá trị `tuổi` tương ứng với các quan sát có giá trị `tiền lương` dưới 250 được thể hiện dưới dạng các điểm màu xám ở phía dưới biểu đồ. Chúng tôi đã thêm một lượng nhiễu nhỏ để làm cho các giá trị `tuổi` dao động nhẹ, sao cho các quan sát có cùng giá trị `tuổi` không che khuất lẫn nhau. Loại biểu đồ này thường được gọi là biểu đồ thảm (rug plot). Để phù hợp với hàm bậc thang, như đã thảo luận trong Phần 7.2, trước tiên chúng ta sử dụng hàm `pd.qcut()` để rời rạc hóa `tuổi` dựa trên các phân vị. Sau đó, chúng ta sử dụng `pd.qcut()`

Sử dụng `pd.get_dummies()` để tạo các cột của ma trận mô hình cho biến phân loại này. Lưu ý rằng hàm này sẽ bao gồm tất cả các cột cho một giá trị đã cho. `pd.get_dummies()` phân loại theo từng cấp độ, thay vì phương pháp thông thường là bỏ đi một trong các cấp độ.

```
Trong [15]: cut_age = pd.qcut(age, 4)
           summarize(sm.OLS(y, pd.get_dummies(cut_age)).fit())
```

Ra ngoài[15]:

	hệ số lỗi chuẩn	t	P> t
(17.999, 33.75]	94.1584 (33.75, 42.0]	1,478 63,692	0.0
116.6608 (42.0, 51.0]	119.1887	1.470 79.385	0.0
(51.0, 80.0]	116.5717	1,416 84,147	0.0
		1,559 74,751	0.0

Ở đây, `pd.qcut()` tự động chọn các điểm cắt dựa trên các phân vị 25%, 50% và 75%, dẫn đến bốn vùng. Chúng ta cũng có thể có Chúng tôi đã chỉ định trực tiếp các phân vị của riêng mình thay vì đối số 4. Đối với các đường cắt Nếu không dựa trên phân vị, chúng ta sẽ sử dụng hàm `pd.cut()`. Hàm `pd.cut()`, `pd.qcut()` (và `pd.cut()`) trả về một biến phân loại có thứ tự. Mô hình hồi quy sau đó tạo ra một tập hợp các biến giả để sử dụng trong hồi quy. Vì `tuổi` là biến duy nhất trong mô hình, giá trị \$94,158.40 là...

mức lương trung bình cho những người dưới 33,75 tuổi và các hệ số khác. Đây là mức lương trung bình của những người thuộc các nhóm tuổi khác. Chúng ta có thể sản xuất Chúng tôi đã đưa ra các dự đoán và vẽ biểu đồ tương tự như trường hợp khớp đa thức.

7.8.2 Đường cong Spline

Để phù hợp với các hàm spline hồi quy, chúng tôi sử dụng các phép biến đổi từ gói `ISLP`. Các hàm đánh giá spline thực tế nằm trong gói `scipy.interpolate`; chúng tôi chỉ đơn giản là gói chúng lại thành các phép biến đổi tương tự như `Poly()` và `PCA()`.

Trong Phần 7.4, chúng ta đã thấy rằng có thể xây dựng các hàm spline hồi quy. một ma trận hàm cơ sở thích hợp. Hàm `BSpline()` tạo ra toàn bộ ma trận hàm cơ sở cho các đường cong spline với tập hợp các hàm được chỉ định. các nút. Theo mặc định, các đường cong B-spline được tạo ra là bậc ba. Để thay đổi bậc, Sử dụng `mức độ tranh luận`.

```
Trong [16]: bs_ = BSpline(internal_knots=[25,40,60], intercept=True).fit(age)
           bs_age = bs_.transform(age)
           bs_age.shape
```

Ra[16]: (3000, 7)

Điều này tạo ra một ma trận bảy cột, đúng như mong đợi đối với cơ sở spline bậc ba với 3 nút bên trong. Chúng ta có thể tạo ra ma trận tương tự bằng cách sử dụng... Đối tượng `bs()`, giúp thêm đối tượng này vào trình xây dựng ma trận mô hình (như trong so sánh `poly()` với hàm `Poly()` (được sử dụng rộng rãi) được mô tả trong Mục 7.8.1. Chúng tôi hiện đang áp dụng mô hình spline bậc ba cho dữ liệu về tiền lương.

```
Trong [17]: bs_age = MS([bs('age', internal_knots=[25,40,60])])
           Xbs = bs_age.fit_transform(Wage)
           M = sm.OLS(y, Xbs).fit()
           tóm tắt(M)
```

```
Ra ngoài[17]:                                     hệ số lỗi chuẩn ...  
điểm chặn 60.494 bs(tuổi, nút)                   9.460 ...  
  
nội bộ=[25, 40, 60]][0] 3.980 12.538 ...  
bs(age, internal_knots=[25, 40, 60])[1] 44.631 bs(age,                9,626 ...  
internal_knots=[25, 40, 60])[2] 62.839 10.755 ...  
bs(age, internal_knots=[25, 40, 60])[3] 55.991 10.706 ...  
bs(age, internal_knots=[25, 40, 60])[4] 50.688 14.402 ...  
bs(age, internal_knots=[25, 40, 60])[5] 16.606 19.126 ...
```

Tên cột hơi rườm rà và khiến chúng tôi phải cắt bớt phần tóm tắt khi in. Có thể thiết lập chúng khi khởi tạo bằng cách sử dụng tên.
Lập luận như sau.

```
Trong [18]: bs_age = MS([bs('age',  
                           internal_knots=[25,40,60],  
                           tên='bs(tuổi)')]))  
  
Xbs = bs_age.fit_transform(Wage)  
M = sm.OLS(y, Xbs).fit()  
tóm tắt(M)
```

```
Ra ngoài[18]:                                     hệ số lỗi chuẩn          t P>|t|  
điểm chặn          60.494 9.460 6.394 0.000  
bs(tuổi, nút)[0] bs(tuổi,          3,981 12,538 0,317 0,751  
nút)[1] bs(tuổi, nút)[2]          44,631 9,626 4,636 0,000  
bs(tuổi, nút)[3] bs(tuổi,          62.839 10.755 5.843 0.000  
nút)[4] bs(tuổi, nút)[5]          55.991 10.706 5.230 0.000  
          50.688 14.402 3.520 0.000  
          16.606 19.126 0.868 0.385
```

Hãy lưu ý rằng có 6 hệ số spline thay vì 7. Điều này là do, bằng cách Theo mặc định, `bs()` giả định `intercept=False`, vì chúng ta thường có một giá trị tổng thể điểm giao nhau trong mô hình. Vì vậy, nó tạo ra cơ sở spline với các điểm nút đã cho. và sau đó loại bỏ một trong các hàm cơ sở để tính đến hệ số chặn.

Chúng ta cũng có thể sử dụng tùy chọn `df` (bậc tự do) để chỉ định độ phức tạp của spline. Như đã thấy ở trên, với 3 nút, cơ sở spline có 6 cột hoặc bậc tự do. Khi chúng ta chỉ định `df=6` thay vì các nút thực tế, `bs()` sẽ tạo ra một đường cong spline với 3 nút được chọn đồng đều. các phân vị của dữ liệu huấn luyện. Chúng ta có thể dễ dàng nhìn thấy những điểm nút được chọn này. Sử dụng trực tiếp hàm `Bspline()` :

```
Trong [19]: Bspline(df=6).fit(age).internal_knots_
```

```
Out[19]: array([33.75, 42.0, 51.0])
```

Khi yêu cầu sáu bậc tự do, phép biến đổi sẽ chọn các điểm nút tại Các độ tuổi 33,75, 42,0 và 51,0, tương ứng với tuổi 25, 50 và 75. phần trăm theo độ tuổi.

Khi sử dụng B-spline, chúng ta không cần phải giới hạn mình chỉ ở các đa thức bậc ba. (tức là bậc = 3). Chẳng hạn, sử dụng bậc = 0 sẽ dẫn đến hằng số từng phần. các hàm, như trong ví dụ với `pd.qcut()` ở trên.

```
Trong [20]: bs_age0 = MS([bs('age',  
                             df=3,  
                             degree=0))].fit(Wage)  
  
Xbs0 = bs_age0.transform(Wage)  
summarize(sm.OLS(y, Xbs0).fit())
```

Ra ngoài[20]:

	hệ số chặn	hệ số lỗi chuẩn	t	P> t
bs(tuổi, df=3, độ=0)[0]	94,158	1.478	63.687	0.0
bs(tuổi, df=3, độ=0)[1]	22.349	2.152	10.388	0.0
bs(tuổi, df=3, độ=0)[2]	24.808	2.044	12.137	0.0
	22.781	2.087	10.917	0.0

Sự phù hợp này nên được so sánh với ô [15] nơi chúng ta sử dụng `qcut()` để tạo ra chia thành bốn nhóm bằng cách cắt ở các phân vị 25%, 50% và 75% của độ tuổi. Vì chúng ta đã chỉ định `df=3` cho các spline bậc không ở đây, cũng sẽ có các điểm nút tại Ba phân vị giống nhau. Mặc dù các hệ số có vẻ khác nhau, nhưng ta thấy rằng Đây là kết quả của cách mã hóa khác nhau. Ví dụ, hệ số đầu tiên là Giống hệt nhau trong cả hai trường hợp, và là giá trị phản hồi trung bình trong khoảng đầu tiên. Đối với Hệ số thứ hai, ta có $94,158 + 22,349 = 116,507 \approx 116,611$, hệ số sau là giá trị trung bình trong ô thứ hai trong ô [15]. Ở đây, hệ số chặn được mã hóa bởi một cột toàn số 1, vì vậy hệ số thứ hai, thứ ba và thứ tư là các bội số của số cộng. Đối với những thùng đó. Tại sao tổng lại không hoàn toàn giống nhau? Hóa ra là...

Hàm `qcut()` sử dụng toán tử `<=`, trong khi hàm `bs()` sử dụng toán tử `<` khi quyết định xem nhóm nào thuộc về nhóm nào.

Để khớp một đường cong spline tự nhiên, chúng ta sử dụng phép biến đổi `NaturalSpline()` với hàm hỗ trợ tương ứng `ns()`. Ở đây, chúng ta khớp một đường cong spline tự nhiên với năm tham số. xác định bậc tự do (không bao gồm hệ số chặn) và vẽ đồ thị kết quả.

Tự nhiên
`Spline()`

Trong [21]:

```
ns_age = MS([ns('age', df=5)]).fit(Wage)
M_ns = sm.OLS(y, ns_age.transform(Wage)).fit()
tóm tắt(M_ns)
```

Ra ngoài[21]:

	hệ số sai số chuẩn	4.708	t	P> t
hệ số chặn	60,475	4.709	12.844	0.000
ns(tuổi, df=5)[0]	61,527	5.717	13.065	0.000
ns(tuổi, df=5)[1]	55,691		9,741	0,000
ns(tuổi, df=5)[2]	46,818	4,948	9,463	0,000
ns(tuổi, df=5)[3]	83.204	11.918	6.982	0.000
ns(tuổi, df=5)[4]	6.877	9,484	0,725	0,468

Bây giờ chúng ta sẽ vẽ đường cong spline tự nhiên bằng hàm vẽ đồ thị của mình.

Trong [22]:

```
plot_wage_fit(age_df,
               ns_age,
               'Đường cong spline tự nhiên, df=5');
```

7.8.3 Làm mịn đường cong Spline và GAM

Spline làm mịn là một trường hợp đặc biệt của GAM với hàm mất mát bình phương sai số. và một đặc trưng duy nhất. Để phù hợp với GAM trong Python, chúng ta sẽ sử dụng gói `pygam`, có thể được cài đặt thông qua `pip install pygam`. Bộ ước lượng `LinearGAM()` sử dụng tổn thất bình phương sai số. GAM được xác định bằng cách liên kết từng cột của ma trận mô hình với một phép toán làm mịn cụ thể: `s` để làm mịn spline; `l` dành cho biến tuyến tính, và `f` dành cho biến phân loại hoặc biến yếu tố. Đối số `0` Tham số được truyền đến `s` bên dưới cho biết rằng bộ làm mịn này sẽ được áp dụng cho cột đầu tiên của một ma trận đặc trưng. Bên dưới, chúng ta truyền vào một ma trận chỉ có một cột: `X_age`. Tham số `lam` là tham số phạt λ như đã thảo luận trong Mục 7.5.2.

pygam
`LinearGAM()`

Trong [23]:

```
X_age = np.asarray(age).reshape((-1,1))
gam = LinearGAM(s_gam(0, lam=0.6))
gam.fit(X_age, y)
```

```
Out[23]: LinearGAM(callbacks=[Deviance(), Diffs()], fit_intercept=True,
               max_iter=100, scale=None, terms=s(0) + intercept, tol=0.0001, verbose=False)
```

Thư viện `pygam` thường mong đợi một ma trận các đặc trưng, vì vậy chúng ta định hình lại biến `'age'` thành một ma trận (mảng hai chiều) thay vì một vectơ (tức là mảng một chiều). Tham số `'-1'` trong lời gọi phương thức `'reshape()'` cho `numpy` biết cần ước tính kích thước của chiều đó dựa trên các phần tử còn lại của bộ giá trị `'shape'`.

Chúng ta hãy cùng tìm hiểu xem sự phù hợp thay đổi như thế nào theo tham số làm mịn `lam`. Hàm `np.logspace()` tương tự như `np.linspace()` nhưng phân bố các điểm `np.logspace()` đều nhau trên thang logarit. Dưới đây, chúng ta thay đổi `lam` từ 10^{-2} đến 10^6 .

```
Trong [24]: fig, ax = subplots(figsize=(8,8))
ax.scatter(age, y, facecolor='gray', alpha=0.5) for lam in
np.logspace(-2, 6, 5): gam = LinearGAM(s_gam(0,
    lam=lam)).fit(X_age, y) ax.plot(age_grid, gam.predict(age_grid),
    label='{:.1e}'.format(lam),
    linewidth=3)

ax.set_xlabel('Tuổi', cỡ chữ=20);
ax.set_ylabel('Lương', cỡ chữ=20);
ax.legend(title='$\lambda$');
```

Gói `pygam` có thể thực hiện tìm kiếm tham số làm mịn tối ưu.

```
Trong [25]: gam_opt = gam.gridsearch(X_age, y) ax.plot(age_grid,
    gam_opt.predict(age_grid),
    label='Tìm kiếm lưới', linewidth=4)
ax.legend() fig
```

Ngoài ra, chúng ta cũng có thể cố định bậc tự do của hàm spline làm mịn bằng cách sử dụng một hàm có sẵn trong gói `ISLP.pygam`. Dưới đây, chúng ta tìm thấy một giá trị của λ cho ta khoảng bốn bậc tự do. Lưu ý rằng các bậc tự do này bao gồm cả hệ số chặn không bị phạt và số hạng tuyến tính của hàm spline làm mịn, do đó vẫn có ít nhất hai bậc tự do.

```
Trong [26]: age_term = gam.terms[0] lam_4
= approx_lam(X_age, age_term, 4) age_term.lam =
lam_4 degrees_of_freedom(X_age,
age_term)
```

```
Out[26]: 4.000000100004728
```

Hãy thay đổi bậc tự do trong một biểu đồ tương tự như trên. Chúng ta chọn bậc tự do bằng bậc tự do mong muốn cộng thêm một để tính đến thực tế là các đường cong spline làm mịn này luôn có một hệ số chặn.

Do đó, giá trị `df` bằng một chỉ đơn giản là sự phù hợp tuyến tính.

```
Trong [27]: fig, ax = subplots(figsize=(8,8))
ax.scatter(X_age,
    y,
```

```

        facecolor='gray',
        alpha=0.3)
for df in [1,3,4,8,15]:
    lam = approx_lam(X_age, age_term, df+1) age_term.lam
    = lam gam.fit(X_age, y)
    ax.plot(age_grid,
            gam.predict(age_grid),
            label='{d}'.format(df),
            linewidth=4)

ax.set_xlabel('Tuổi', fontsize=20);
ax.set_ylabel('Lương', fontsize=20);
ax.legend(title='Bậc tự do');

```

Mô hình cộng với nhiều thuật ngữ

Ưu điểm của các mô hình cộng tính tổng quát nằm ở khả năng phù hợp với các mô hình hồi quy đa biến với tính linh hoạt cao hơn so với các mô hình tuyến tính. Chúng tôi trình bày hai phương pháp: phương pháp đầu tiên theo cách thủ công hơn bằng cách sử dụng các hàm spline tự nhiên và các hàm hằng từng phần, và phương pháp thứ hai sử dụng gói `pygam` và các hàm spline làm mịn.

Bây giờ chúng tôi tự tay điều chỉnh GAM để dự đoán **tiền lương** bằng cách sử dụng các hàm spline tự nhiên của **năm** và **tuổi**, coi **giáo dục** là một yếu tố dự báo định tính, như trong (7.16). Vì đây chỉ là một mô hình hồi quy tuyến tính lớn sử dụng lựa chọn hàm cơ sở phù hợp, chúng ta có thể thực hiện điều này một cách đơn giản bằng cách sử dụng hàm `sm.OLS()`.

Ở đây, chúng ta sẽ xây dựng ma trận mô hình theo cách thủ công hơn, vì chúng ta muốn truy cập các thành phần riêng biệt khi xây dựng biểu đồ phụ thuộc từng phần.

```

Trong [28]: ns_age = NaturalSpline(df=4).fit(age)
ns_year = NaturalSpline(df=5).fit(Wage['year'])
Xs = [ns_age.transform(age),
      ns_year.transform(Wage['year']),
      pd.get_dummies(Wage['education']).values]
X_bh = np.hstack(Xs) gam_bh
= sm.OLS(y, X_bh).fit()

```

Ở đây, hàm `NaturalSpline()` là hàm chủ lực hỗ trợ hàm trợ giúp `ns()`. Chúng tôi chọn sử dụng tất cả các cột của ma trận chỉ báo cho biến phân loại **giáo dục**, do đó hệ số chặn trở nên dư thừa. Cuối cùng, chúng tôi xếp chồng ba ma trận thành phần theo chiều ngang để tạo thành ma trận mô hình `X_bh`.

Bây giờ chúng ta sẽ trình bày cách xây dựng biểu đồ phụ thuộc từng phần cho mỗi thành phần trong mô hình GAM cơ bản của chúng ta. Chúng ta có thể làm điều này bằng tay, với các lưới dữ liệu về **tuổi** và **năm**. Chúng ta chỉ cần dự đoán với các ma trận `X` mới, sửa tất cả các đặc trưng ngoại trừ một đặc trưng tại một thời điểm.

```

Trong [29]: age_grid = np.linspace(age.min(), age.max(),
                                  100)

X_age_bh = X_bh.copy()[1:100]
X_age_bh[:,0] = X_bh[:,0].mean(0)[None,:]
X_age_bh[:,4] = ns_age.transform(age_grid) preds =
gam_bh.get_prediction(X_age_bh) bounds_age =
preds.conf_int(alpha=0.05)

```

```
partial_age = preds.predicted_mean center =
partial_age.mean() partial_age -= center
bounds_age -= center fig, ax =
subplots(figsize=(8,8))
ax.plot(age_grid, partial_age, 'b', linewidth=3)
ax.plot(age_grid, bounds_age[:,0], 'r--', linewidth=3) ax.plot(age_grid,
bounds_age[:,1], 'r--', linewidth=3) ax.set_xlabel('Tuổi') ax.set_ylabel('Ảnh
hưởng đến tiền lương') ax.set_title(' Sự phụ thuộc một phần của tuổi vào tiền
lương', fontsize=20);
```

Hãy giải thích chi tiết hơn những gì chúng ta đã làm ở trên. Ý tưởng là tạo ra một ma trận dự đoán mới, trong đó tất cả các cột ngoại trừ cột liên quan đến **tuổi** đều là hằng số (và được đặt bằng giá trị trung bình của dữ liệu huấn luyện). Bốn cột dành cho **tuổi** được điền bằng cơ sở spline tự nhiên được đánh giá tại 100 giá trị trong **age_grid**.

1. Chúng tôi đã tạo một lưới có độ dài 100 theo **chiều tuổi**, và tạo một ma trận **X_age_bh** với 100 hàng và cùng số cột như ma trận **X_bh**.
2. Chúng tôi đã thay thế mỗi hàng của ma trận này bằng giá trị trung bình của các cột nguyên bản.
3. Sau đó, chúng ta chỉ thay thế bốn cột đầu tiên biểu thị **tuổi** bằng cơ sở spline tự nhiên được tính toán tại các giá trị trong **age_grid**.

Các bước còn lại chắc hẳn đã quen thuộc rồi.

Chúng tôi cũng xem xét ảnh hưởng của yếu tố **năm** đến **tiền lương**; quy trình tương tự.

```
Trong [30]: year_grid = np.linspace(2003, 2009, 100)
year_grid = np.linspace(Wage['year'].min(), Wage['year'].max(),
100)

X_year_bh = X_bh.copy()[1:100]
X_year_bh[:] = X_bh[:].mean(0)[None,:]
X_year_bh[:,4:9] = ns_year.transform(year_grid) preds =
gam_bh.get_prediction(X_year_bh) bounds_year =
preds.conf_int(alpha=0.05) partial_year = preds.predicted_mean
center = partial_year.mean() partial_year -= center
bounds_year -= center fig, ax =
subplots(figsize=(8,8))
ax.plot(year_grid, partial_year ,
'b', linewidth=3) ax.plot(year_grid,
bounds_year[:,0], 'r--', linewidth=3) ax.plot(year_grid, bounds_year[:,1],
'r--', linewidth=3) ax.set_xlabel('Năm') ax.set_ylabel('Ảnh hưởng đến tiền lương')
ax.set_title(' Sự phụ thuộc một phần của năm lương', cỡ chữ=20);
```

Giờ đây, chúng ta điều chỉnh mô hình (7.16) bằng cách sử dụng spline làm mịn thay vì spline tự nhiên. Tất cả các thuật ngữ trong (7.16) được điều chỉnh đồng thời, xem xét lẫn nhau để giải thích phản hồi. Gói **pygam** chỉ hoạt động với ma trận, vì vậy chúng ta phải chuyển đổi chuỗi phân loại **education** thành dạng mảng, có thể tìm thấy bằng thuộc tính **cat.codes** của **education**. Vì **year** chỉ có 7 giá trị duy nhất, chúng ta chỉ sử dụng bảy hàm cơ sở cho nó.


```
Trong [31]: gam_full = LinearGAM(s_gam(0) + s_gam(1,
                                     n_splines=7) + f_gam(2, lam=0))

Xgam = np.column_stack([age, Wage['year'],
                                     Wage['education'].cat.codes]) gam_full =
gam_full.fit(Xgam, y)
```

Hai thuật ngữ `s_gam()` tạo ra các đường cong spline làm mịn và sử dụng giá trị mặc định cho λ ($\text{lam}=0,6$), giá trị này khá tùy ý. Đối với thuật ngữ phân loại `giáo dục`, được chỉ định bằng thuật ngữ `f_gam()`, chúng ta chỉ định $\text{lam}=0$ để tránh bất kỳ sự co rút nào. Chúng ta tạo ra biểu đồ phụ thuộc từng phần theo `tuổi` để thấy được tác động của những lựa chọn này.

Các giá trị cho đồ thị được tạo ra bởi gói `pygam`. Chúng tôi cung cấp hàm `plot_gam()` cho các đồ thị phụ thuộc một phần trong `ISLP.pygam`, điều này giúp công việc này dễ dàng hơn so với ví dụ trước của chúng tôi với các đường cong spline tự nhiên. `plot_gam()`

```
Trong [32]: fig, ax = subplots(figsize=(8,8))
plot_gam(gam_full, 0, ax=ax)
ax.set_xlabel('Tuổi')
ax.set_ylabel('Ảnh hưởng đến tiền lương')
ax.set_title('Sự phụ thuộc một phần của tuổi vào tiền lương - mặc định lam=0.6',
             cở chữ=20);
```

Ta thấy hàm số hơi ngoằn ngoèo. Việc chỉ định bậc `tự do (df)` sẽ tự nhiên hơn là chỉ định giá trị cho lam . Ta điều chỉnh lại mô hình GAM bằng cách sử dụng bốn bậc tự do cho `tuổi` và `năm`. Hãy nhớ rằng việc cộng thêm một ở dưới đây đã tính đến hệ số chặn của đường cong spline làm mịn.

```
Trong [33]: age_term = gam_full.terms[0] age_term.lam
           = approx_lam(Xgam, age_term, df=4+1) year_term = gam_full.terms[1]
year_term.lam = approx_lam(Xgam, year_term,
                           df=4+1) gam_full = gam_full.fit(Xgam, y)
```

Lưu ý rằng việc cập nhật `age_term.lam` ở trên cũng cập nhật nó trong `gam_full.terms[0]` ! Tương tự đối với `year_term.lam`.

Khi lặp lại đồ thị theo `độ tuổi`, ta thấy nó mượt mà hơn nhiều. Ta cũng lặp đồ thị theo `năm`.

```
Trong [34]: fig, ax = subplots(figsize=(8,8)) plot_gam(gam_full,
1, ax=ax) ax.set_xlabel('Năm')

ax.set_ylabel('Ảnh hưởng đến tiền
lương') ax.set_title('Sự phụ thuộc một phần của
năm vào tiền lương', fontsize=20)
```

Cuối cùng, chúng ta vẽ biểu đồ về `trình độ học vấn`, một biến phân loại. Biểu đồ phụ thuộc từng phần này khác biệt và phù hợp hơn với tập hợp các hằng số được ước tính cho mỗi cấp độ của biến này.

```
Trong [35]: fig, ax = subplots(figsize=(8, 8)) ax =
plot_gam(gam_full, 2) ax.set_xlabel('Giáo
dục') ax.set_ylabel('Ảnh hưởng đến
tiền lương')
```

```
ax.set_title(' Sự phụ thuộc một phần của tiền lương vào trình độ học vấn',
             fontsize=20);
ax.set_xticklabels(Wage['education'].cat.categories, fontsize=8);
```

Kiểm định ANOVA cho các mô hình cộng tính

Trong tất cả các mô hình của chúng ta, hàm số của **năm** có vẻ khá tuyến tính. Chúng ta có thể thực hiện một loạt các phép thử ANOVA để xác định mô hình nào trong ba mô hình này là tốt nhất: mô hình GAM không bao gồm **năm** (M1), mô hình GAM sử dụng hàm tuyến tính của **năm** (M2) hoặc mô hình GAM sử dụng hàm spline của **năm** (M3).

```
Trong [36]: gam_0 = LinearGAM(age_term + f_gam(2, lam=0)) gam_0.fit(Xgam, y)
gam_linear =
LinearGAM(age_term +
           l_gam(1, lam=0) +
           f_gam(2, lam=0))
gam_linear.fit(Xgam, y)
```

```
Out[36]: LinearGAM(callbacks=[Deviance(), Diffs()], fit_intercept=True,
                    max_iter=100, scale=None, terms=s(0) + l(1) + f(2) + intercept, tol=0.0001, verbose=False)
```

Hãy chú ý việc chúng ta sử dụng **age_term** trong các biểu thức ở trên. Chúng ta làm điều này vì trước đó chúng ta đã đặt giá trị cho **lam** trong thuật ngữ này để đạt được bốn bậc tự do.

Để đánh giá trực tiếp ảnh hưởng của **năm**, chúng tôi tiến hành phân tích ANOVA trên ba biến số.

Các mẫu xe phù hợp với kích thước phía trên.

```
Trong [37]: anova_gam(gam_0, gam_linear, gam_full)
```

Ra ngoài[37]:

	sự lệch lạc	df	deviance	diff	df_diff	Giá trị p của F
0	3714362.366	2991.004		NaN		NaN
1	3696745.823	2990.005	17616.543		0,999 14,265	0,002
2	3693142.930	2987.007	3602.894		2,998 0,972	0,436

Chúng tôi nhận thấy có bằng chứng thuyết phục cho thấy mô hình GAM với hàm tuyến tính theo **năm** tốt hơn mô hình GAM không bao gồm **biến năm** (giá trị p = 0,002). Tuy nhiên, không có bằng chứng nào cho thấy cần thiết phải sử dụng hàm phi tuyến tính theo **năm** (giá trị p = 0,435). Nói cách khác, dựa trên kết quả của phân tích ANOVA này, mô hình M2 được ưu tiên hơn.

Chúng ta có thể lặp lại quy trình tương tự đối với **tuổi tác**. Chúng ta thấy có bằng chứng rất rõ ràng cho thấy cần có một số hạng phi tuyến tính đối với **tuổi tác**.

```
Trong [38]: gam_0 = LinearGAM(year_term + f_gam(2,
           lam=0)) gam_linear =
LinearGAM(l_gam(0, lam=0) +
           năm_kỳ hạn +
           f_gam(2, lam=0))
gam_0.fit(Xgam, y)
gam_linear.fit(Xgam, y)
anova_gam(gam_0, gam_linear, gam_full)
```

```
Ra ngoài[38]:          sự lệch lạc          df deviance_diff df_diff          Giá trị p của F
0 3975443.045 2991.001          NaN          NaN          NaN          NaN
1 3850246.908 2990.001          125196.137          1.000 101.270 0.000
2 3693142.930 2987.007          157103.978          2.993 42.448 0.000
```

Có một phương thức `summary()` (chi tiết) cho kết quả khớp GAM. (Chúng tôi không (Sao chép nó vào đây.)

```
Trong [39]: gam_full.summary()
```

Chúng ta có thể đưa ra dự đoán từ các đối tượng `gam`, giống như từ các đối tượng `lm`, bằng cách sử dụng Phương thức `predict()` cho lớp `gam`. Tại đây, chúng ta thực hiện dự đoán về Bộ dữ liệu huấn luyện.

```
Trong [40]: Yhat = gam_full.predict(Xgam)
```

Để phù hợp với mô hình GAM hồi quy logistic, chúng ta sử dụng hàm `LogisticGAM()` từ thư viện `LogisticGAM(). pygam`.

```
Trong [41]: gam_logit = LogisticGAM(age_term +
                                l_gam(1, lam=0) +
                                f_gam(2, lam=0))
gam_logit.fit(Xgam, high_earn)
```

```
Out[41]: LogisticGAM(callbacks=[Deviance(), Diffs(), Accuracy()],
                    fit_intercept=True, max_iter=100,
                    terms=s(0) + l(1) + f(2) + intercept, tol=0.0001, verbose=False)
```

```
Trong [42]: fig, ax = subplots(figsize=(8, 8))
ax = plot_gam(gam_logit, 2)
ax.set_xlabel('Giáo dục')
ax.set_ylabel('Ảnh hưởng đến tiền lương')
ax.set_title(' Sự phụ thuộc một phần của tiền lương vào trình độ học vấn',
             cở chữ=20);
ax.set_xticklabels(Wage['education'].cat.categories, fontsize=8);
```

Mô hình có vẻ rất phẳng, với các thanh lỗi đặc biệt cao đối với...
Hạng mục đầu tiên. Chúng ta hãy xem xét dữ liệu kỹ hơn một chút.

```
Trong [43]: pd.crosstab(Wage['high_earn'], Wage['education'])
```

Chúng ta thấy rằng không có người nào có thu nhập cao trong nhóm trình độ học vấn đầu tiên.
Điều đó có nghĩa là mô hình sẽ khó khớp. Chúng ta sẽ sử dụng mô hình hậu cần.
Hồi quy GAM loại trừ tất cả các quan sát thuộc loại này. Điều này
Mang lại kết quả hợp lý hơn.

Để làm vậy, chúng ta có thể chọn một tập con của ma trận mô hình, mặc dù điều này sẽ không loại bỏ cột từ `Xgam`. Trong khi chúng ta có thể suy ra cột nào tương ứng với
Để đảm bảo tính khả thi về mặt tái tạo, chúng tôi định hình lại ma trận mô hình trên tính năng này.
Tập hợp con nhỏ hơn.

```
Trong [44]: only_hs = Wage['education'] == '1. < HS Grad'
Wage_ = Wage.loc[ only_hs]
Xgam_ = np.column_stack([Wage_['age'],
                        Lương_['năm'],
                        Wage_['education'].cat.codes -1])
high_earn_ = Wage_['high_earn']
```

Ở dòng áp chót phía trên, chúng ta trừ đi một từ các mã của danh mục do lỗi trong `pygam`. Việc này chỉ đơn giản là đổi tên các giá trị giáo dục và do đó không ảnh hưởng đến độ phù hợp.

Bây giờ chúng ta đã khớp mô hình.

```
Trong [45]: gam_logit_ = LogisticGAM(age_term +
                                     year_term +
                                     f_gam(2, lam=0))
gam_logit_.fit(Xgam_, high_earn_)
```

```
Out[45]: LogisticGAM(callbacks=[Deviance(), Diffs(), Accuracy()],
    fit_intercept=True, max_iter=100, terms=s(0) +
    s(1) + f(2) + intercept, tol=0.0001, verbose=False)
```

Hãy cùng xem xét ảnh hưởng của trình độ học vấn, năm học và tuổi tác đến vị thế người có thu nhập cao. Giờ thì chúng ta đã loại bỏ những quan sát đó rồi.

```
Trong [46]: fig, ax = subplots(figsize=(8, 8)) ax =
    plot_gam(gam_logit_, 2) ax.set_xlabel('Giáo
    dục') ax.set_ylabel('Ảnh hưởng đến
    tiền lương') ax.set_title(' Sự phụ thuộc một
    phần của tình trạng người có thu nhập cao vào giáo dục ')
    ', cỡ chữ=20);
ax.set_xticklabels(Wage['education'].cat.categories[1:], fontsize=8);
```

```
Trong [47]: fig, ax = subplots(figsize=(8, 8)) ax =
    plot_gam(gam_logit_, 1) ax.set_xlabel('Năm')
    ax.set_ylabel('Ảnh hưởng đến
    tiền lương') ax.set_title(' Sự phụ thuộc một
    phần của trạng thái người có thu nhập cao vào năm', fontsize=20);
```

```
Trong [48]: fig, ax = subplots(figsize=(8, 8)) ax =
    plot_gam(gam_logit_, 0)
    ax.set_xlabel('Tuổi')
    ax.set_ylabel('Ảnh hưởng đến tiền lương')
    ax.set_title(' Sự phụ thuộc một phần của trạng thái người kiếm tiền cao vào tuổi',
    cỡ chữ=20);
```

7.8.4 Hồi quy cục bộ

Chúng tôi minh họa việc sử dụng hồi quy cục bộ bằng cách sử dụng hàm `lowess()` từ `sm.nonparametric`. Một số triển khai của GAM cho phép các thuật ngữ là các toán tử hồi quy cục bộ; điều này không đúng trong `pygam`.

Ở đây, chúng tôi sử dụng mô hình hồi quy tuyến tính cục bộ với khoảng cách 0,2 và 0,5; nghĩa là, mỗi vùng lân cận bao gồm 20% hoặc 50% số quan sát. Như dự đoán, sử dụng khoảng cách 0,5 cho kết quả mượt mà hơn so với 0,2.

```
Trong [49]: lowess = sm.nonparametric.lowess
fig, ax = subplots(figsize=(8,8)) ax.scatter(age,
y, facecolor='gray', alpha=0.5) for span in [0.2, 0.5]: fitted =
    lowess(y,
```

```

        ax.plot(age_grid,
                fitted,
                label='{:.1f}'.format(span),
                linewidth=4)

ax.set_xlabel('Tuổi', cỡ chữ=20);
ax.set_ylabel('Lương', cỡ chữ=20);
ax.legend(title='span', cỡ chữ=15);

```

7.9 Bài tập

Khái niệm

- Trong chương này đã đề cập rằng có thể thu được một spline hồi quy bậc ba với một điểm nút tại ξ bằng cách sử dụng cơ sở có dạng $x, x^2, x^3, (x - \xi)^3$ trong đó $(x - \xi)^3 = (x - \xi)^3$ nếu $x > \xi$ và bằng 0 nếu ngược lại. Bây giờ chúng ta sẽ chứng minh rằng một hàm có dạng

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 (x - \xi)^3_+$$

Đây thực sự là một hàm spline hồi quy bậc ba, bất kể giá trị của $\beta_0, \beta_1, \beta_2, \beta_3, \beta_4$ là bao nhiêu.

- (a) Tìm một đa thức bậc ba

$$f_1(x) = a_1 + b_1 x + c_1 x^2 + d_1 x^3$$

sao cho $f(x) = f_1(x)$ với mọi $x \leq \xi$. Biểu diễn a_1, b_1, c_1, d_1 theo $\beta_0, \beta_1, \beta_2, \beta_3, \beta_4$.

- (b) Tìm một đa thức bậc ba

$$f_2(x) = a_2 + b_2 x + c_2 x^2 + d_2 x^3$$

sao cho $f(x) = f_2(x)$ với mọi $x > \xi$. Biểu diễn a_2, b_2, c_2, d_2 theo $\beta_0, \beta_1, \beta_2, \beta_3, \beta_4$. Chúng ta đã xác định được $f(x)$ là một đa thức từng phần. (c) Chứng

- minh rằng $f_1(\xi) = f_2(\xi)$. Tức là, $f(x)$ liên tục tại ξ . (d) Chứng minh rằng $f_1'(\xi) = f_2'(\xi)$. Tức là, $f(x)$ liên tục tại ξ . (e) Chứng minh rằng $f_1''(\xi) = f_2''(\xi)$. Tức là, $f(x)$ liên tục tại ξ .

Do đó, $f(x)$ thực sự là một hàm spline bậc ba.

Gợi ý: Phần (d) và (e) của bài toán này yêu cầu kiến thức về giải tích một biến. Để nhắc lại, cho một đa thức bậc ba

$$f_1(x) = a_1 + b_1 x + c_1 x^2 + d_1 x^3,$$

đạo hàm bậc nhất có dạng

$$f_1'(x) = b_1 + 2c_1 x + 3d_1 x^2$$



và đạo hàm bậc hai có dạng

$$f''(x) = 2c_1 + 6d_1x.$$

2. Giả sử một đường cong $\hat{g}$ được tính toán để khớp mượt mà với một tập hợp n điểm bằng cách sử dụng công thức sau:

$$\hat{g} = \arg \min_g \sum_{i=1}^n (y_i - g(x_i))^2 + \lambda \int_0^1 g'(x)^2 dx,$$

trong đó $g(m)$ biểu thị đạo hàm bậc m của g (và $g(0) = g$). Cung cấp các ví dụ minh họa về $\hat{g}$ trong mỗi trường hợp sau đây.

(a) $\lambda = \infty, m = 0$. (b)

$\lambda = \infty, m = 1$. (c) $\lambda =$

$\infty, m = 2$. (d) $\lambda = \infty, m$

$= 3$. (e) $\lambda = 0, m = 3$.

3. Giả sử ta khớp một đường cong với các hàm cơ sở $b_1(X) = X$, $b_2(X) = (X - 1)2I(X \geq 1)$. (Lưu ý rằng $I(X \geq 1)$ bằng 1 khi $X \geq 1$ và bằng 0 trong các trường hợp khác.) Ta khớp mô hình hồi quy tuyến tính

$$Y = \beta_0 + \beta_1 b_1(X) + \beta_2 b_2(X),$$

và thu được các ước lượng hệ số $\hat{\beta}_0 = 1$, $\hat{\beta}_1 = 1$, $\hat{\beta}_2 = 2$. Vẽ đồ thị ước lượng giữa $X = -2$ và $X = 2$. Ghi chú các điểm cắt trục tung, hệ số góc và các thông tin liên quan khác.

4. Giả sử chúng ta khớp một đường cong với các hàm cơ sở $b_1(X) = I(0 \leq X \leq 2)$ ($(X - 1)I(1 \leq X \leq 2)$), $b_2(X) = (X - 3)I(3 \leq X \leq 4) + I(4 < X \leq 5)$.

Chúng tôi đã áp dụng mô hình hồi quy tuyến tính.

$$Y = \beta_0 + \beta_1 b_1(X) + \beta_2 b_2(X),$$

và thu được các ước lượng hệ số $\hat{\beta}_0 = 1$, $\hat{\beta}_1 = 1$, $\hat{\beta}_2 = 3$. Vẽ đồ thị ước lượng giữa $X = -2$ và $X = 6$. Ghi chú các điểm cắt trục tung, hệ số góc và các thông tin liên quan khác.

5. Hãy xem xét hai đường cong, $\hat{g}^1$ và $\hat{g}^2$, được định nghĩa bởi

$$\hat{g}^1 = \arg \min_g \sum_{i=1}^n (y_i - g(x_i))^2 + \lambda \int_0^1 g'(x)^2 dx,$$

$$\hat{g}^2 = \arg \min_g \sum_{i=1}^n (y_i - g(x_i))^2 + \lambda \int_0^1 g'(4x)^2 dx,$$

trong đó $g(m)$ biểu thị đạo hàm bậc m của g .

(a) Khi $\lambda \rightarrow \infty$, $\hat{g}^1$ hay $\hat{g}^2$ sẽ có RSS huấn luyện nhỏ hơn?

(b) Khi $\lambda \rightarrow \infty$, $\hat{g}^1$ hay $\hat{g}^2$ sẽ có RSS kiểm tra nhỏ hơn?

(c) Với $\lambda = 0$, $\hat{g}^1$ hay $\hat{g}^2$ sẽ có cả RSS huấn luyện và kiểm tra nhỏ hơn?

Đã áp dụng

6. Trong bài tập này, bạn sẽ phân tích sâu hơn bộ dữ liệu về **tiền lương** đã được xem xét. xuyên suốt chương này.

- (a) Thực hiện hồi quy đa thức để dự đoán **mức lương** dựa trên **tuổi**. Sử dụng phương pháp kiểm định chéo để chọn bậc d tối ưu cho đa thức. Bậc nào đã được chọn, và kết quả này so sánh như thế nào với kết quả kiểm định giả thuyết bằng ANOVA? Vẽ đồ thị biểu diễn sự phù hợp của đa thức với dữ liệu. (b) Sử dụng hàm bậc thang để dự đoán **mức lương** dựa trên **tuổi**, và thực hiện kiểm định chéo để

chọn số lượng bậc cắt tối ưu. Vẽ đồ thị biểu diễn sự phù hợp thu được.

7. Tập dữ liệu **Tiền lương** chứa một số đặc điểm khác chưa được đề cập trong chương này, chẳng hạn như tình trạng hôn nhân (**maritl**), loại công việc (**jobclass**) và các yếu tố khác. Hãy khám phá mối quan hệ giữa một số yếu tố dự báo khác này và **tiền lương**, đồng thời sử dụng các kỹ thuật khớp phi tuyến tính để xây dựng các mô hình linh hoạt cho dữ liệu. Tạo biểu đồ thể hiện kết quả thu được và viết tóm tắt các phát hiện của bạn.

8. Hãy áp dụng một số mô hình phi tuyến tính đã được nghiên cứu trong chương này vào tập dữ liệu **Auto**. Có bằng chứng nào cho thấy mối quan hệ phi tuyến tính trong tập dữ liệu này không? Hãy tạo một số biểu đồ minh họa để chứng minh câu trả lời của bạn.

9. Câu hỏi này sử dụng các biến **dis** (trung bình có trọng số của khoảng cách đến trung tâm việc làm ở Boston) và **nox** (nồng độ oxit nitơ tính bằng phần triệu) từ dữ liệu của **Boston**. Chúng ta sẽ coi **dis** là biến dự báo và **nox** là biến phản hồi.

- (a) Sử dụng hàm **poly()** từ mô-đun **ISLP.models** để thực hiện hồi quy đa thức bậc ba nhằm dự đoán **nox** dựa trên **dis**. Báo cáo kết quả hồi quy và vẽ đồ thị dữ liệu thu được cùng với đường cong hồi quy đa thức.

- (b) Vẽ đồ thị các đường khớp đa thức cho một loạt các bậc đa thức khác nhau (ví dụ, từ 1 đến 10), và báo cáo tổng bình phương phần dư tương ứng.

- (c) Thực hiện kiểm định chéo hoặc một phương pháp khác để chọn bậc tối ưu cho đa thức và giải thích kết quả của bạn.

- (d) Sử dụng hàm **bs()** từ mô-đun **ISLP.models** để khớp một spline hồi quy nhằm dự đoán **nox** bằng cách sử dụng **dis**. Báo cáo kết quả khớp với bốn bậc tự do. Bạn đã chọn các điểm nút như thế nào? Vẽ đồ thị kết quả khớp. (e) Bây giờ hãy khớp một spline hồi quy với một loạt các bậc tự do, vẽ đồ thị kết

quả khớp và báo cáo RSS thu được. Mô tả các kết quả thu được.

- (f) Thực hiện kiểm định chéo hoặc một phương pháp khác để chọn bậc tự do tốt nhất cho hàm spline hồi quy trên dữ liệu này. Hãy mô tả kết quả của bạn.

10. Câu hỏi này liên quan đến tập dữ liệu của trường Cao đẳng .

(a) Chia dữ liệu thành tập huấn luyện và tập kiểm tra. Sử dụng học phí ngoài tiểu bang làm biến phản hồi và các biến khác làm biến dự đoán, thực hiện lựa chọn từng bước tiến lên trên tập huấn luyện để xác định một mô hình thỏa đáng chỉ sử dụng một tập con các biến dự đoán.

(b) Huấn luyện mô hình GAM trên dữ liệu huấn luyện, sử dụng học phí ngoài tiểu bang làm biến phản hồi và các đặc trưng đã chọn ở bước trước làm biến dự đoán. Vẽ đồ thị kết quả và giải thích các phát hiện của bạn. (c) Đánh giá mô hình thu được trên tập dữ liệu kiểm tra và giải thích các kết quả thu được.

(d) Đối với những biến số nào, nếu có, có bằng chứng về mối quan hệ phi tuyến tính. Mối liên hệ với câu trả lời là gì?

11. Trong Mục 7.7, đã đề cập rằng các mô hình GAM thường được xây dựng bằng phương pháp backfitting . Ý tưởng đằng sau backfitting thực ra khá đơn giản. Bây giờ chúng ta sẽ tìm hiểu về backfitting trong bối cảnh hồi quy tuyến tính đa biến.

Giả sử chúng ta muốn thực hiện hồi quy tuyến tính đa biến, nhưng lại không có phần mềm để làm điều đó. Thay vào đó, chúng ta chỉ có phần mềm để thực hiện hồi quy tuyến tính đơn giản. Do đó, chúng ta áp dụng phương pháp lặp lại sau: chúng ta liên tục giữ cố định tất cả các ước lượng hệ số ngoại trừ một ước lượng, và chỉ cập nhật ước lượng hệ số đó bằng hồi quy tuyến tính đơn giản. Quá trình này được tiếp tục cho đến khi hội tụ—nghĩa là, cho đến khi các ước lượng hệ số ngừng thay đổi.

Bây giờ chúng ta sẽ thử nghiệm điều này trên một ví dụ đơn giản.

(a) Tạo một biến phản hồi Y và hai biến dự đoán X_1 và X_2 , với $n = 100$.

(b) Viết một hàm `simple_reg()` nhận hai đối số là `outcome` và `feature`, thực hiện việc khớp một mô hình hồi quy tuyến tính đơn giản với `outcome` và `feature` này, và trả về hệ số chặn và hệ số góc ước tính. (c) Khởi tạo `beta1` với một giá trị do bạn chọn. Giá trị bạn chọn không quan trọng. (d) Giữ `beta1` cố định, sử dụng hàm `simple_reg()` của bạn để khớp mô hình.

người mẫu:

$$Y = \text{beta1} \cdot X_1 = \beta_0 + \beta_2 X_2 + \epsilon$$

Lưu các giá trị thu được dưới dạng `beta0` và `beta2`. (e)

Giữ `beta2` cố định, tiến hành điều chỉnh mô hình.

$$Y = \text{beta2} \cdot X_2 = \beta_0 + \beta_1 X_1 + \epsilon$$

Lưu kết quả dưới dạng `beta0` và `beta1` (ghi đè lên các giá trị trước đó).

(f) Viết một vòng lặp `for` để lặp lại (c) và (d) 1.000 lần. Báo cáo các giá trị ước tính của `beta0`, `beta1` và `beta2` tại mỗi lần lặp của vòng lặp `for`. Tạo một đồ thị hiển thị từng giá trị này, cùng với `beta0`, `beta1` và `beta2`.

(g) So sánh câu trả lời của bạn ở (e) với kết quả của việc thực hiện hồi quy tuyến tính bội để dự đoán Y bằng cách sử dụng X_1 và X_2 .

(e) Sử dụng phương thức `axline()` để chồng các ước tính hệ số hồi quy tuyến tính bội đó lên biểu đồ thu được trong (e). (h)

Trên tập dữ liệu này, cần bao nhiêu lần lặp backfitting để có được một xấp xỉ "tốt" cho các ước tính hệ số hồi quy bội?

12. Bài toán này là phần tiếp theo của bài tập trước. Trong một ví dụ đơn giản với $p = 100$, hãy chứng minh rằng ta có thể xấp xỉ các ước lượng hệ số hồi quy tuyến tính bội bằng cách thực hiện lặp đi lặp lại hồi quy tuyến tính đơn giản trong quy trình hiệu chỉnh ngược. Cần bao nhiêu lần lặp hiệu chỉnh ngược để có được một ước lượng "tốt" cho các ước lượng hệ số hồi quy bội? Hãy vẽ đồ thị để minh họa câu trả lời của bạn.

8

Phương pháp dựa trên cây



Trong chương này, chúng ta mô tả các phương pháp dựa trên cây cho hồi quy và phân loại. Các phương pháp này bao gồm việc phân tầng hoặc phân đoạn không gian dự đoán thành các nhóm nhỏ hơn, số lượng các vùng đơn giản. Để đưa ra dự đoán cho một quan sát nhất định, chúng ta thường sử dụng giá trị phản hồi trung bình hoặc giá trị phản hồi phổ biến nhất cho các quan sát huấn luyện trong khu vực mà nó thuộc về. Vì tập hợp của các quy tắc phân chia được sử dụng để phân đoạn không gian dự đoán có thể được tóm tắt như sau: Một cái cây, những loại phương pháp này được gọi là phương pháp cây quyết định.

cây quyết định

Các phương pháp dựa trên cây rất đơn giản và hữu ích cho việc diễn giải. Tuy nhiên, về mặt dự đoán, chúng thường không cạnh tranh được với các phương pháp học có giám sát tốt nhất, chẳng hạn như những phương pháp được trình bày trong Chương 6 và 7. độ chính xác. Do đó, trong chương này chúng ta cũng giới thiệu về bagging, rừng ngẫu nhiên, boosting và cây hồi quy cộng tính Bayes. Mỗi phương pháp này đều có những đặc điểm riêng. Quá trình này bao gồm việc tạo ra nhiều cây con, sau đó kết hợp chúng lại để tạo thành một cây duy nhất. dự đoán đồng thuận. Chúng ta sẽ thấy rằng việc kết hợp một số lượng lớn cây thường có thể dẫn đến những cải thiện đáng kể về độ chính xác dự đoán, ở mức độ chi phí cho một số tổn thất trong quá trình giải thích.

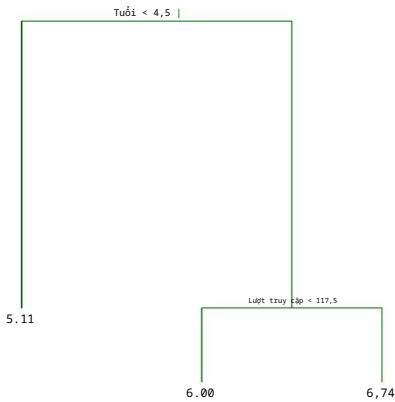
8.1 Khái niệm cơ bản về cây quyết định

Cây quyết định có thể được áp dụng cho cả bài toán hồi quy và bài toán phân loại. Trước tiên, chúng ta xem xét các bài toán hồi quy, sau đó chuyển sang bài toán phân loại.

8.1.1 Cây hồi quy

Để làm rõ hơn về cây hồi quy, chúng ta bắt đầu với một ví dụ đơn giản.

hồi quy
cây



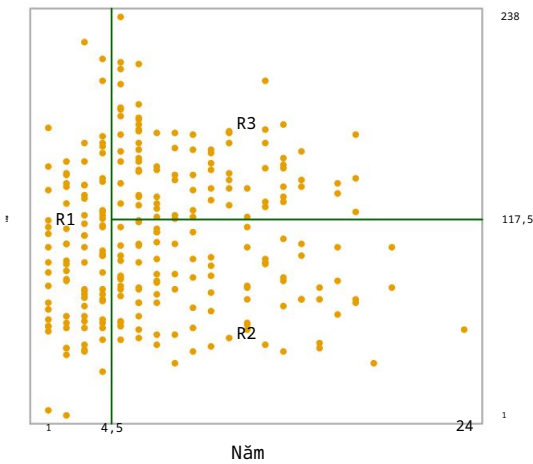
HÌNH 8.1. Đối với dữ liệu **Hitters** , cây hồi quy được sử dụng để dự đoán logarit mức lương của một cầu thủ bóng chày, dựa trên số năm anh ta chơi ở giải đấu lớn và số lần đánh trúng đích mà anh ta thực hiện trong năm trước đó. Tại một nút bên trong nhất định, nhãn (có dạng $X_j < tk$) biểu thị nhánh bên trái xuất phát từ điểm phân nhánh đó, và nhánh bên phải tương ứng với $X_j \geq tk$. Ví dụ, sự phân nhánh ở đỉnh cây tạo ra hai nhánh lớn. Nhánh bên trái tương ứng với các năm $< 4,5$, và nhánh bên phải tương ứng với các năm $\geq 4,5$. Cây có hai nút bên trong và ba nút cuối cùng, hay còn gọi là lá. Con số trong mỗi lá là giá trị trung bình của phản hồi đối với các quan sát nằm ở đó.

Dự đoán mức lương của các cầu thủ bóng chày bằng cách sử dụng cây hồi quy

Chúng tôi sử dụng tập dữ liệu **Hitters** để dự đoán mức lương của một cầu thủ bóng chày dựa trên Số năm (số năm anh ta chơi ở giải đấu lớn) và Số lần đánh trúng bóng (số lần anh ta đánh trúng bóng trong năm trước). Đầu tiên, chúng tôi loại bỏ các quan sát thiếu giá trị Lương , và biến đổi logarit mức lương để phân bố của nó có hình dạng chuông điển hình hơn. (Hãy nhớ rằng mức lương được đo bằng nghìn đô la.)

Hình 8.1 thể hiện cây hồi quy được xây dựng phù hợp với dữ liệu này. Nó bao gồm một loạt các quy tắc phân nhánh, bắt đầu từ đỉnh của cây. Phân nhánh trên cùng gán các quan sát có Năm $< 4,5$ vào nhánh bên trái. Mức lương dự đoán cho những người chơi này được đưa ra bởi giá trị phản hồi trung bình cho những người chơi trong tập dữ liệu có Năm $< 4,5$. Đối với những người chơi như vậy, logarit mức lương trung bình là 5,107, và do đó chúng ta đưa ra dự đoán là 5,107 nghìn đô la, tức là 165.174 đô la, cho những người chơi này. Những người chơi có Năm $\geq 4,5$ được gán vào nhánh bên phải, và sau đó nhóm đó được chia nhỏ hơn nữa theo Số lần đánh trúng. Nhìn chung, cây phân tầng hoặc phân đoạn người chơi thành ba vùng không gian dự đoán: người chơi đã chơi bốn năm trở xuống, người chơi đã chơi năm năm trở lên và có ít hơn 118 lần đánh trúng trong năm ngoái, và người chơi đã chơi năm năm trở lên và có ít nhất 118 lần đánh trúng trong năm ngoái. Ba vùng này có thể được viết là $R_1 = \{X \mid R_1 = \{X \mid \text{Năm} \leq 4,5\}, R_2 = \{X \mid \text{Năm} \geq 4,5, \text{Lượt truy cập} < 117,5\}$, và $R_3 = \{X \mid \text{Năm} \geq 4,5, \text{Lượt truy cập} \geq 117,5\}$. Hình 8.2 minh họa điều này .

1Cả Năm và Số Lượt Nghe đều là số nguyên trong dữ liệu này; hàm được sử dụng để phù hợp với cây này. Đánh dấu các điểm chia tại điểm giữa của hai giá trị liên kề.



HÌNH 8.2. Phân vùng ba khu vực cho tập dữ liệu Hitters từ cây hồi quy được minh họa trong Hình 8.1.

các khu vực được biểu diễn theo Năm và Số lượt truy cập. Mức lương dự đoán cho ba nhóm này lần lượt là $\$1,000 \times 5.107 = \$165,174$, $\$1,000 \times 5.999 = \$402,834$ và $\$1,000 \times 6.740 = \$845,346$.

Theo phép so sánh với cây, các vùng R1, R2 và R3 được gọi là các nút cuối hoặc lá của cây. Như trong Hình 8.1, cây quyết định thường được vẽ ngược, theo nghĩa là các lá nằm ở dưới cùng của cây. Các điểm dọc theo cây nơi không gian dự đoán được phân chia được gọi là các nút bên trong. Trong Hình 8.1, hai nút bên trong được chỉ ra bằng văn bản $\text{Years} < 4.5$ và $\text{Hits} < 117.5$. Chúng ta gọi các đoạn cây nối các nút này là các nhánh.

phần cuối
nút
lá cây
nút bên
trong
chi nhánh

Chúng ta có thể diễn giải sơ đồ hồi quy được hiển thị trong Hình 8.1 như sau: Số năm kinh nghiệm là yếu tố quan trọng nhất trong việc xác định mức lương, và những cầu thủ ít kinh nghiệm hơn sẽ nhận được mức lương thấp hơn so với những cầu thủ giàu kinh nghiệm hơn. Vì một cầu thủ ít kinh nghiệm, số lần đánh trúng bóng mà anh ta thực hiện trong năm trước dường như không ảnh hưởng nhiều đến mức lương của anh ta. Nhưng đối với những cầu thủ đã chơi ở giải đấu lớn từ năm năm trở lên, số lần đánh trúng bóng thực hiện trong năm trước có ảnh hưởng đến mức lương, và những cầu thủ thực hiện nhiều lần đánh trúng bóng hơn trong năm ngoái thường có mức lương cao hơn. Sơ đồ hồi quy được hiển thị trong Hình 8.1 có thể là sự đơn giản hóa quá mức mối quan hệ thực sự giữa Số lần đánh trúng bóng, Số năm kinh nghiệm và Mức lương. Tuy nhiên, nó có những ưu điểm so với các loại mô hình hồi quy khác (như những mô hình đã thấy trong Chương 3 và 6): nó dễ diễn giải hơn và có hình ảnh đồ họa đẹp mắt.

Dự đoán thông qua phân tầng không gian đặc trưng

Bây giờ chúng ta sẽ thảo luận về quy trình xây dựng cây hồi quy. Nói một cách khái quát, có hai bước.

1. Chúng ta chia không gian dự đoán – tức là tập hợp các giá trị có thể có cho $X_1, X_2, \dots, X_p$ – thành J vùng riêng biệt và không chồng chéo, $R_1, R_2, \dots, R_J$.

2. Đối với mỗi quan sát nằm trong vùng R_j , chúng ta đưa ra cùng một dự đoán, đó đơn giản là giá trị trung bình của các giá trị phản hồi cho các quan sát huấn luyện trong vùng R_j .

Ví dụ, giả sử ở Bước 1 ta thu được hai vùng, R_1 và R_2 , và giá trị trung bình phản hồi của các quan sát huấn luyện trong vùng thứ nhất là 10, trong khi giá trị trung bình phản hồi của các quan sát huấn luyện trong vùng thứ hai là 20. Khi đó, với một quan sát $X = x$, nếu $x \in R_1$ ta sẽ dự đoán giá trị là 10, và nếu $x \in R_2$ ta sẽ dự đoán giá trị là 20.

Bây giờ chúng ta sẽ đi sâu hơn vào Bước 1 ở trên. Làm thế nào để xây dựng các vùng $R_1, \dots, R_J$? Về lý thuyết, các vùng có thể có bất kỳ hình dạng nào. Tuy nhiên, chúng ta chọn chia không gian dự đoán thành các hình chữ nhật hoặc hình hộp có chiều cao để đơn giản hóa và dễ dàng diễn giải mô hình dự đoán thu được. Mục tiêu là tìm các hình hộp $R_1, \dots, R_J$ sao cho giảm thiểu RSS, được cho bởi

$$\sum_{j=1}^J \sum_{i \in R_j} (y_i - \bar{y}^{R_j})^2, \tag{8.1}$$

trong đó $\bar{y}^{R_j}$ là giá trị phản hồi trung bình cho các quan sát huấn luyện trong ô thứ j . Thật không may, việc xem xét mọi phân vùng có thể có của không gian đặc trưng thành J ô là không khả thi về mặt tính toán. Vì lý do này, chúng tôi sử dụng phương pháp tham lam từ trên xuống, được gọi là phân tách nhị phân đệ quy. Phương pháp này là từ trên xuống vì nó bắt đầu từ đỉnh của cây (tại điểm này tất cả các quan sát thuộc về một vùng duy nhất) và sau đó lần lượt phân tách không gian dự đoán; mỗi lần phân tách được biểu thị bằng hai nhánh mới ở phía dưới cây. Nó là phương pháp tham lam vì ở mỗi bước của quá trình xây dựng cây, phép phân tách tốt nhất được thực hiện ở bước cụ thể đó, thay vì nhìn về phía trước và chọn một phép phân tách sẽ dẫn đến một cây tốt hơn trong một bước nào đó trong tương lai.

phân tách
nhị
phân đệ quy

Để thực hiện phân tách nhị phân đệ quy, trước tiên chúng ta chọn bộ dự đoán X_j và điểm cắt s sao cho việc chia không gian dự đoán thành các vùng $\{X|X_j < s\}$ và $\{X|X_j \geq s\}$ dẫn đến sự giảm RSS lớn nhất có thể. (Ký hiệu $\{X|X_j < s\}$ có nghĩa là vùng không gian dự đoán trong đó X_j nhận giá trị nhỏ hơn s .) Nghĩa là, chúng ta xem xét tất cả các bộ dự đoán $X_1, \dots, X_p$ và tất cả các giá trị có thể có của điểm cắt s cho mỗi bộ dự đoán, sau đó chọn bộ dự đoán và điểm cắt sao cho cây kết quả có RSS thấp nhất. Cụ thể hơn, đối với bất kỳ j và s nào, chúng ta định nghĩa cặp nửa mặt phẳng.

$$R_1(j, s) = \{X|X_j < s\} \text{ và } R_2(j, s) = \{X|X_j \geq s\}, \tag{8.2}$$

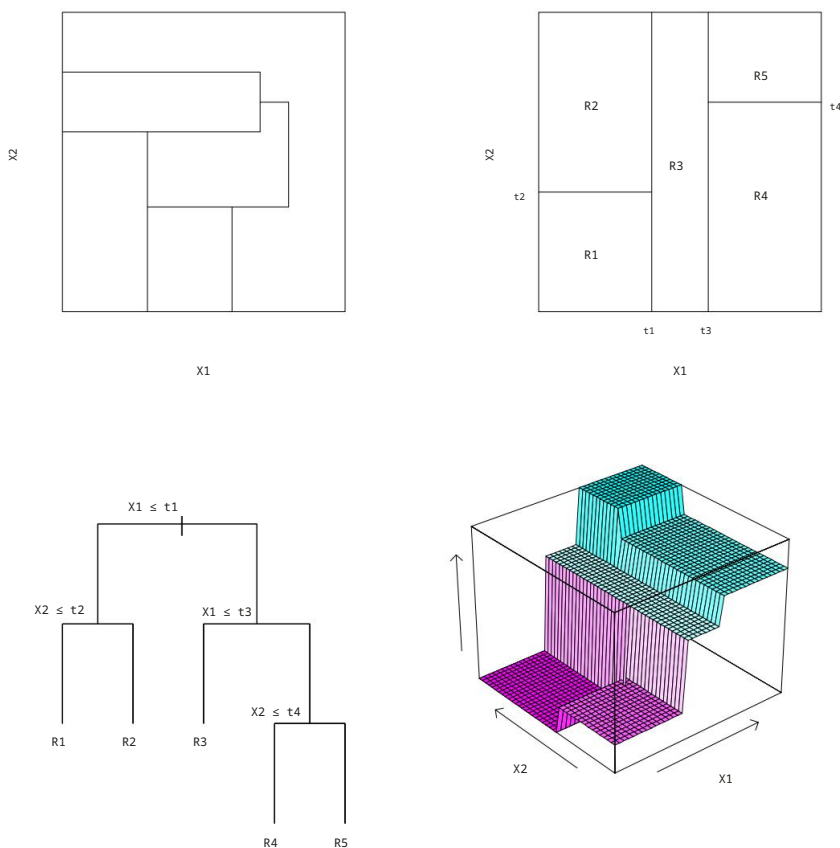
và chúng ta tìm giá trị của j và s sao cho phương trình đạt giá trị nhỏ nhất.

$$\sum_{i: x_i \in R_1(j,s)} (y_i - \bar{y}^{R_1(j,s)})^2 + \sum_{i: x_i \in R_2(j,s)} (y_i - \bar{y}^{R_2(j,s)})^2, \tag{8.3}$$

trong đó $\bar{y}^{R_1}$ là phản hồi trung bình cho các quan sát huấn luyện trong $R_1(j, s)$ và $\bar{y}^{R_2}$ là phản hồi trung bình cho các quan sát huấn luyện trong $R_2(j, s)$.

Việc tìm ra các giá trị của j và s sao cho (8.3) được tối thiểu hóa có thể được thực hiện khá nhanh chóng, đặc biệt khi số lượng đặc trưng p không quá lớn.

Tiếp theo, chúng ta lặp lại quy trình, tìm kiếm biến dự đoán tốt nhất và điểm cắt tốt nhất để chia dữ liệu thêm nữa nhằm giảm thiểu RSS trong phạm vi đó.



HÌNH 8.3. Trên cùng bên trái: Một phân vùng không gian đặc trưng hai chiều không thể thu được từ phép phân tách nhị phân đệ quy. Trên cùng bên phải: Kết quả của phép phân tách nhị phân đệ quy trên một ví dụ hai chiều. Dưới cùng bên trái: Một cây tương ứng với phân vùng trong hình trên cùng bên phải. Dưới cùng bên phải: Một biểu đồ phối cảnh của bề mặt dự đoán tương ứng với cây đó.

mỗi vùng kết quả. Tuy nhiên, lần này, thay vì chia toàn bộ không gian dự đoán, chúng ta chia một trong hai vùng đã được xác định trước đó.

Hiện tại chúng ta có ba vùng. Một lần nữa, chúng ta tìm cách chia nhỏ một trong ba vùng này hơn nữa để giảm thiểu RSS. Quá trình này tiếp tục cho đến khi đạt được tiêu chí dừng; ví dụ, chúng ta có thể tiếp tục cho đến khi không có vùng nào chứa nhiều hơn năm quan sát.

Sau khi các vùng $R_1, \dots, R_J$ được tạo ra, chúng ta dự đoán phản hồi cho một quan sát thử nghiệm nhất định bằng cách sử dụng giá trị trung bình của các quan sát huấn luyện trong vùng mà quan sát thử nghiệm đó thuộc về.

Hình 8.3 minh họa một ví dụ về cách tiếp cận này với năm vùng khác nhau.

Cắt tia cây

Quy trình được mô tả ở trên có thể tạo ra các dự đoán tốt trên tập dữ liệu huấn luyện, nhưng có khả năng dẫn đến hiện tượng quá khớp dữ liệu, gây ra hiệu suất kém trên tập dữ liệu kiểm tra. Điều này là do cây kết quả có thể quá phức tạp. Một cây nhỏ hơn

Việc sử dụng ít nhánh hơn (tức là ít vùng $R_1, \dots, R_J$ hơn) có thể dẫn đến phương sai thấp hơn và khả năng diễn giải tốt hơn với chi phí là một chút sai lệch. Một phương án thay thế khả thi cho quy trình được mô tả ở trên là chỉ xây dựng cây cho đến khi sự giảm RSS do mỗi nhánh vượt quá một ngưỡng (cao) nào đó. Chiến lược này sẽ tạo ra những cây nhỏ hơn, nhưng lại quá thiên cận vì một sự phân nhánh tưởng chừng như vô ích ở giai đoạn đầu của cây có thể được theo sau bởi một sự phân nhánh rất có lợi-nghĩa là, một sự phân nhánh dẫn đến việc giảm đáng kể RSS sau này.

Do đó, một chiến lược tốt hơn là xây dựng một cây rất lớn T_0 , rồi cắt tỉa nó để thu được một cây con. Làm thế nào để xác định cách cắt tỉa cây tốt nhất? Theo trực giác, mục tiêu của chúng ta là chọn một cây con dẫn đến tỷ lệ lỗi kiểm thử thấp nhất. Với một cây con đã cho, chúng ta có thể ước tính lỗi kiểm thử của nó bằng cách sử dụng phương pháp kiểm định chéo hoặc phương pháp tập dữ liệu xác thực. Tuy nhiên, việc ước tính lỗi kiểm định chéo cho mọi cây con có thể có sẽ quá phức tạp, vì số lượng cây con có thể có là cực kỳ lớn.

Thay vào đó, chúng ta cần một cách để chọn ra một tập hợp nhỏ các cây con để xem xét.

Cắt tỉa theo độ phức tạp chi phí-còn được gọi là cắt tỉa mất xích yếu nhất-cung cấp cho chúng ta một cách để làm điều này. Thay vì xem xét mọi cây con có thể có, chúng ta xem xét một chuỗi các cây được lập chỉ mục bởi một tham số điều chỉnh không âm q . Với mỗi giá trị của α , tương ứng có một cây con T_α sao cho

$$|T|$$
$$\sum_{m=1}^M (y_m - \hat{y}_m)^2 + \alpha |T|$$

$m=1 \text{ i: } x_i \text{ Rm}$

cắt tỉa

cây con

cắt

tỉa chi phí phức

cắt tỉa mất xích

yếu nhất

(8.4)

là nhỏ nhất có thể. Ở đây $|T|$ biểu thị số lượng nút cuối của cây T , R_m là hình chữ nhật (tức là tập con của không gian dự đoán) tương ứng với nút cuối thứ m , và $\hat{y}_m$ là phản hồi dự đoán liên quan đến R_m -nghĩa là, giá trị trung bình của các quan sát huấn luyện trong R_m .

Tham số điều chỉnh α kiểm soát sự cân bằng giữa độ phức tạp của cây con và độ phù hợp của nó với dữ liệu huấn luyện. Khi $\alpha = 0$, thì cây con T sẽ đơn giản bằng T_0 , bởi vì khi đó (8.4) chỉ đo lỗi huấn luyện.

Tuy nhiên, khi α tăng lên, sẽ có một cái giá phải trả cho việc có một cây với nhiều nút cuối cùng, và do đó, đại lượng (8.4) sẽ có xu hướng được tối thiểu hóa đối với một cây con nhỏ hơn. Phương trình 8.4 gợi nhớ đến lasso (6.7) từ Chương 6, trong đó một công thức tương tự đã được sử dụng để kiểm soát độ phức tạp của một mô hình tuyến tính.

Hóa ra khi chúng ta tăng α từ 0 trong (8.4), các nhánh sẽ bị cắt tỉa khỏi cây theo cách lồng nhau và có thể dự đoán được, do đó việc thu được toàn bộ chuỗi cây con như một hàm của α rất dễ dàng. Chúng ta có thể chọn một giá trị của α bằng cách sử dụng tập dữ liệu xác thực hoặc sử dụng phương pháp kiểm định chéo. Sau đó, chúng ta quay lại tập dữ liệu đầy đủ và thu được cây con tương ứng với α . Quá trình này được tóm tắt trong Thuật toán 8.1.

Hình 8.4 và 8.5 hiển thị kết quả của việc xây dựng và cắt tỉa cây hồi quy trên dữ liệu **Hitters**, sử dụng chín đặc trưng. Đầu tiên, chúng tôi chia ngẫu nhiên tập dữ liệu thành hai nửa, thu được 132 quan sát trong tập huấn luyện và 131 quan sát trong tập kiểm tra. Sau đó, chúng tôi xây dựng một cây hồi quy lớn trên dữ liệu huấn luyện và thay đổi α trong (8.4) để tạo ra các cây con với số lượng nút cuối khác nhau. Cuối cùng, chúng tôi thực hiện kiểm định chéo sáu lần để ước tính MSE kiểm định chéo của cây.

Thuật toán 8.1 Xây dựng cây hồi quy

1. Sử dụng phương pháp phân tách nhị phân đệ quy để xây dựng một cây lớn trên dữ liệu huấn luyện, chỉ dừng lại khi mỗi nút cuối cùng có ít hơn một số lượng quan sát tối thiểu nào đó.

2. Áp dụng phương pháp cắt tia dựa trên độ phức tạp chi phí cho cây lớn để thu được một chuỗi các cây con tốt nhất, như một hàm của α .

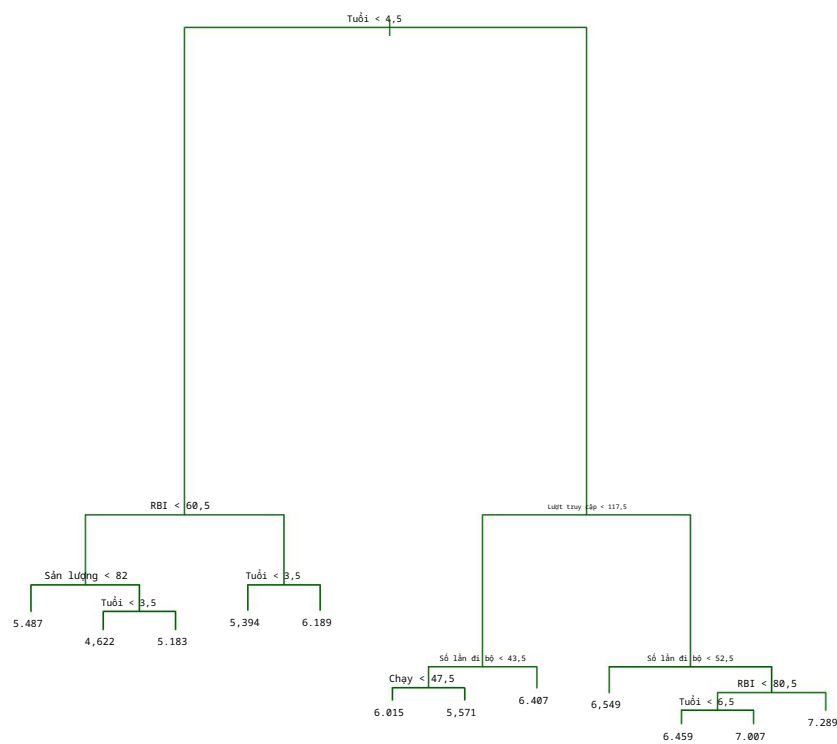
3. Sử dụng phương pháp kiểm định chéo K-fold để chọn α . Tức là, chia tập dữ liệu huấn luyện thành K fold. Với mỗi $k = 1, \dots, K$:

(a) Lặp lại Bước 1 và 2 trên tất cả các phần dữ liệu huấn luyện ngoại trừ phần thứ k.

(b) Đánh giá sai số dự đoán bình phương trung bình trên dữ liệu trong phần thứ k bị loại bỏ, như một hàm của α .

Tính trung bình các kết quả cho mỗi giá trị của α , và chọn α sao cho sai số trung bình là nhỏ nhất.

4. Trả về cây con từ Bước 2 tương ứng với giá trị đã chọn của α .
- một hàm của α . (Chúng tôi chọn thực hiện kiểm định chéo sáu lần vì 132 là bội số chính xác của sáu.) Cây hồi quy chưa được cắt tia được hiển thị trong Hình 8.4. Đường cong màu xanh lá cây trong Hình 8.5 cho thấy lỗi CV là một hàm của số lượng lá, 2 trong khi đường cong màu cam biểu thị lỗi kiểm định. Các thanh lỗi chuẩn xung quanh các lỗi ước tính cũng được hiển thị.
- Để tham khảo, đường cong lỗi huấn luyện được hiển thị bằng màu đen. Lỗi CV là một phép xấp xỉ hợp lý cho lỗi kiểm tra: lỗi CV đạt giá trị nhỏ nhất đối với cây ba nút, trong khi lỗi kiểm tra cũng giảm xuống ở cây ba nút (mặc dù nó đạt giá trị thấp nhất ở cây mười nút).
- Hình 8.1 minh họa cây đã được tia cạnh với ba nút cuối cùng .
- 8.1.2 Cây phân loại
- Cây phân loại rất giống với cây hồi quy, ngoại trừ việc nó được sử dụng để dự đoán phản hồi định tính thay vì định lượng. Nhớ lại rằng đối với cây hồi quy, phản hồi dự đoán cho một quan sát được đưa ra bởi phản hồi trung bình của các quan sát huấn luyện thuộc cùng một nút cuối. Ngược lại, đối với cây phân loại, chúng ta dự đoán rằng mỗi quan sát thuộc về lớp xuất hiện phổ biến nhất trong các quan sát huấn luyện trong vùng mà nó thuộc về. Khi diễn giải kết quả của cây phân loại, chúng ta thường quan tâm không chỉ đến dự đoán lớp tương ứng với một vùng nút cuối cụ thể, mà còn đến tỷ lệ lớp trong số các quan sát huấn luyện nằm trong vùng đó.
- phân loại
cây
- Nhiệm vụ xây dựng cây phân loại khá giống với nhiệm vụ xây dựng cây hồi quy. Cũng như trong trường hợp hồi quy, chúng ta sử dụng phương pháp đệ quy.
- 2Mặc dù lỗi CV được tính toán như một hàm của α , nhưng việc hiển thị kết quả như một hàm của $|T|$, số lượng lá, sẽ thuận tiện hơn; điều này dựa trên mối quan hệ giữa α và $|T|$ trong cây ban đầu được xây dựng từ tất cả dữ liệu huấn luyện.



HÌNH 8.4. Phân tích cây hồi quy cho dữ liệu Hitters . Cây chưa được cắt tỉa.
Kết quả thu được từ việc phân tách tham lam từ trên xuống trên dữ liệu huấn luyện được thể hiện trong hình.

Phân tách nhị phân để xây dựng cây phân loại. Tuy nhiên, trong phân loại
Trong cài đặt này, RSS không thể được sử dụng làm tiêu chí để thực hiện việc phân tách nhị phân.
Một giải pháp thay thế tự nhiên cho RSS là tỷ lệ lỗi phân loại. Vì chúng tôi dự định
gán một quan sát trong một khu vực nhất định cho biến thể xuất hiện thường xuyên nhất. phân loại
tỷ lệ lỗi
Trong nhóm các quan sát huấn luyện ở vùng đó, tỷ lệ lỗi phân loại là
đơn giản là tỷ lệ các quan sát huấn luyện trong khu vực đó không
thuộc loại phổ biến nhất:

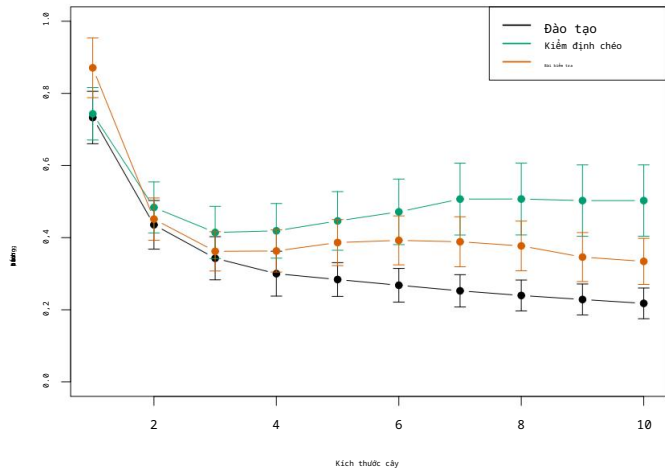
$$E = 1 - \max_k (\hat{p}_{mk}). \tag{8.5}$$

Ở đây, $\hat{p}_{mk}$ biểu thị tỷ lệ các quan sát huấn luyện trong thứ m .
vùng thuộc lớp thứ k . Tuy nhiên, hóa ra phân loại
lỗi này không đủ nhạy để phát triển cây quyết định, và trên thực tế, hai lỗi khác cũng vậy.
Các biện pháp là điều nên làm.

Chỉ số Gini được định nghĩa bởi Chỉ số Gini

$$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk}), \tag{8.6}$$

một thước đo tổng phương sai trên K lớp. Không khó để nhận thấy
Chỉ số Gini có giá trị nhỏ nếu tất cả các $\hat{p}_{mk}$ đều gần với
không hoặc một. Vì lý do này, chỉ số Gini được coi là thước đo của



HÌNH 8.5. Phân tích cây hồi quy cho dữ liệu **Hitters** . Quá trình huấn luyện, Kết quả kiểm định chéo và MSE kiểm thử được hiển thị dưới dạng hàm số của số lượng thiết bị đầu cuối. các nút trong cây đã được cắt tỉa. Các dải sai số chuẩn được hiển thị. Giá trị tối thiểu Lỗi kiểm định chéo xảy ra khi kích thước cây là ba.

Độ tinh khiết của nút – giá trị nhỏ cho biết nút đó chủ yếu chứa Những quan sát từ một lớp học duy nhất.

Một phương án thay thế cho chỉ số Gini là entropy, được định nghĩa bởi entropy

$$D = \sum_{k=1}^K p^*_k \log p^*_k. \tag{8.7}$$

Vì $0 \leq p^*_k \leq 1$, nên suy ra $0 \leq -p^*_k \log p^*_k$. Có thể chứng minh rằng Độ entropy sẽ có giá trị gần bằng 0 nếu tất cả các p^*_k đều gần bằng 0 hoặc gần bằng một. Do đó, giống như chỉ số Gini, entropy sẽ có giá trị nhỏ. giá trị nếu nút thứ m là thuần túy. Trên thực tế, hóa ra chỉ số Gini và Về mặt số học, entropy của chúng khá giống nhau.

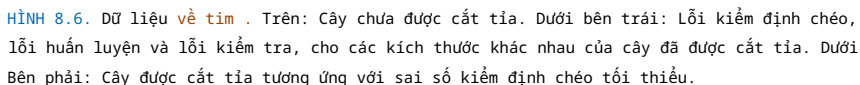
Khi xây dựng cây phân loại, người ta thường sử dụng chỉ số Gini hoặc entropy. thường được sử dụng để đánh giá chất lượng của một lần chia tách cụ thể, vì những điều này Hai phương pháp này nhạy cảm hơn với độ tinh khiết của nút so với phân loại. tỷ lệ lỗi. Bất kỳ một trong ba phương pháp này đều có thể được sử dụng khi cắt tỉa cây. cây quyết định, nhưng tỷ lệ lỗi phân loại sẽ tốt hơn nếu độ chính xác dự đoán của

Mục tiêu cuối cùng là có được cây đã được tỉa cành hoàn chỉnh.

Hình 8.6 minh họa một ví dụ trên tập dữ liệu **Tim** . Dữ liệu này chứa kết quả nhị phân HD cho 303 bệnh nhân có triệu chứng đau ngực. Giá trị kết quả "Có" cho biết có bệnh tim dựa trên...

Xét nghiệm chụp mạch máu, trong khi "Không" có nghĩa là không mắc bệnh tim. Có 13 yếu tố dự báo bao gồm Tuổi, Giới tính, Chol (chỉ số đo cholesterol) và các bệnh tim mạch khác. và các phép đo chức năng phổi. Kết quả kiểm định chéo tạo ra một cây với sáu nhánh. các nút đầu cuối.

Trong phần thảo luận cho đến nay, chúng ta đã giả định rằng các biến dự báo có giá trị liên tục. Tuy nhiên, cây quyết định có thể được xây dựng ngay cả khi có sự hiện diện của các biến dự báo định tính. Ví dụ, trong Dữ liệu về tim , một số yếu tố dự báo, chẳng hạn như Giới tính, Thal (xét nghiệm gắng sức Thallium),



Hình 8.6 có một đặc điểm đáng ngạc nhiên: một số phép chia tạo ra hai các nút cuối có cùng giá trị dự đoán. Ví dụ, hãy xem xét nhánh `RestECG<1` bị tách ra gần phía dưới bên phải của cây chưa được cắt tỉa. Bất kể Dưa trên giá trị của `RestECG`, giá trị phản hồi "Có" được dự đoán cho những người quan sát được...

quan sát. Vậy thì tại sao lại phải thực hiện phép chia? Phép chia được thực hiện vì nó dẫn đến tăng độ tinh khiết của nút. Nghĩa là, tất cả 9 quan sát Tương ứng với chiếc lá bên phải có giá trị phản hồi là **Có**, trong khi đó 7/11 trong số những trường hợp tương ứng với lá bên trái có giá trị phản hồi là **Vâng**. Tại sao độ tinh khiết của nút lại quan trọng? Giả sử chúng ta có một quan sát thử nghiệm thuộc vùng được xác định bởi nút lá bên phải đó. Khi đó chúng ta có thể khá chắc chắn rằng giá trị phản hồi của nó là **Có**. Ngược lại, nếu một bài kiểm tra Nếu quan sát thuộc vùng được chỉ định bởi lá bên trái, thì giá trị phản hồi của nó có lẽ là **Có**, nhưng chúng ta không chắc chắn lắm. Mặc dù Việc tách **RestECG<1** không làm giảm lỗi phân loại, mà cải thiện... Chỉ số Gini và entropy là hai chỉ số nhạy cảm hơn với độ tinh khiết của nút mạng.

8.1.3 Cây quyết định so với mô hình tuyến tính

Cây hồi quy và cây phân loại có "hướng vị" rất khác so với các loại cây khác. Các phương pháp cổ điển cho hồi quy và phân loại được trình bày trong Chương 3. và 4. Cụ thể, hồi quy tuyến tính giả định một mô hình có dạng

$$f(X) = \beta_0 + \sum_{j=1}^p x_j \beta_j, \quad (8.8)$$

trong khi đó, cây hồi quy giả định một mô hình có dạng

$$f(X) = \sum_{m=1}^M c_m \cdot 1(X \in R_m) \quad (8.9)$$

trong đó $R_1, \dots, R_M$ biểu thị sự phân vùng không gian đặc trưng, như trong Hình 8.3.

Mô hình nào tốt hơn? Điều đó phụ thuộc vào vấn đề cụ thể. Nếu mối quan hệ giữa các đặc trưng và phản hồi được xấp xỉ tốt bởi

một mô hình tuyến tính như trong (8.8), thì một phương pháp như hồi quy tuyến tính sẽ Phương pháp này có khả năng hoạt động tốt và sẽ vượt trội hơn các phương pháp khác như cây hồi quy. Điều đó không khai thác cấu trúc tuyến tính này. Nếu thay vào đó có mối quan hệ phi tuyến tính và phức tạp giữa các đặc điểm và phản hồi như

như được chỉ ra bởi mô hình (8.9), thì cây quyết định có thể vượt trội hơn các phương pháp cổ điển. Một ví dụ minh họa được hiển thị trong Hình 8.7. Tương đối Hiệu suất của các phương pháp dựa trên cây và các phương pháp cổ điển có thể được đánh giá bằng cách ước tính lỗi kiểm thử, sử dụng phương pháp kiểm định chéo hoặc tập dữ liệu kiểm định. phương pháp tiếp cận (Chương 5).

Đĩ nhiên, ngoài lỗi kiểm thử đơn thuần, còn có những yếu tố khác cần xem xét.

Việc lựa chọn phương pháp học thống kê đóng vai trò quan trọng; ví dụ, trong một số trường hợp nhất định, dự đoán bằng cây quyết định có thể được ưu tiên hơn vì tính dễ hiểu và trực quan hóa.

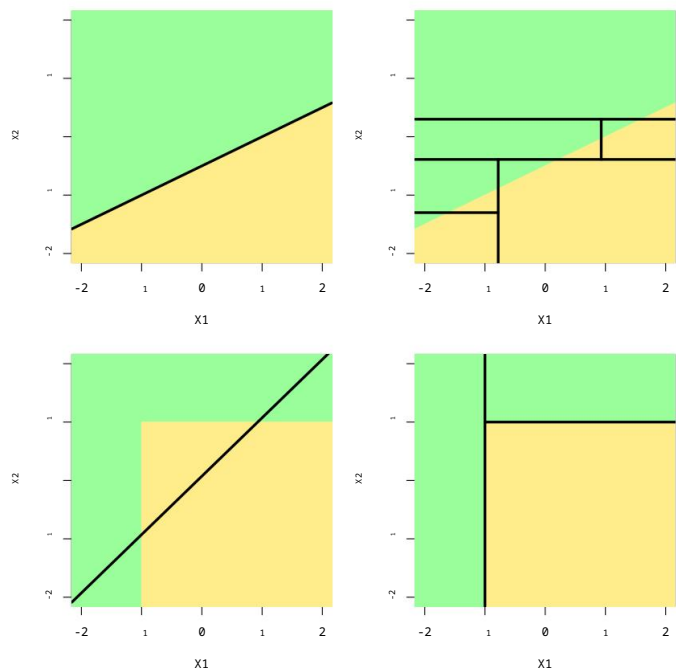
8.1.4 Ưu điểm và nhược điểm của cây

Cây quyết định dùng trong hồi quy và phân loại có một số ưu điểm.

so với các phương pháp cổ điển hơn được trình bày trong Chương 3 và 4:

Cây cối rất dễ giải thích cho mọi người. Trên thực tế, chúng thậm chí còn dễ hơn nữa.

Để giải thích rõ hơn so với hồi quy tuyến tính!



HÌNH 8.7. Hàng trên: Một ví dụ phân loại hai chiều trong đó Ranh giới quyết định thực sự là tuyến tính và được biểu thị bằng các vùng tô bóng. Một ví dụ kinh điển Phương pháp giả định ranh giới tuyến tính (bên trái) sẽ cho kết quả tốt hơn cây quyết định. thực hiện các phép chia song song với các trục (bên phải). Hàng dưới: Ở đây, ranh giới quyết định thực sự là phi tuyến tính. Ở đây, mô hình tuyến tính không thể nắm bắt được ranh giới thực sự. Đường ranh giới quyết định (bên trái), trong khi cây quyết định là một cây thành công (bên phải).

Một số người tin rằng cây quyết định phản ánh hành vi con người một cách chính xác hơn. việc ra quyết định hiệu quả hơn so với các phương pháp hồi quy và phân loại. đã được đề cập trong các chương trước.

Cây có thể được hiển thị dưới dạng đồ họa và dễ dàng được hiểu ngay cả bởi người không chuyên. người không chuyên (đặc biệt nếu họ còn nhỏ).

Cây quyết định có thể dễ dàng xử lý các biến dự báo định tính mà không cần phải... Tạo các biến giả.

Thật không may, nhìn chung cây cối không có khả năng dự đoán ở mức độ tương tự. độ chính xác tương đương với một số phương pháp hồi quy và phân loại khác. được thấy trong cuốn sách này.

Ngoài ra, cây cối có thể rất yếu ớt. Nói cách khác, một cây nhỏ Sự thay đổi trong dữ liệu có thể gây ra sự thay đổi lớn trong ước tính cuối cùng. cây.

Tuy nhiên, bằng cách tổng hợp nhiều cây quyết định, sử dụng các phương pháp như bagging, Với rừng ngẫu nhiên và thuật toán boosting, hiệu suất dự đoán của cây quyết định có thể được... Đã được cải thiện đáng kể. Chúng tôi sẽ giới thiệu các khái niệm này trong phần tiếp theo.

8.2 Phương pháp Bagging, Rừng ngẫu nhiên, Boosting và Cây hồi quy cộng tính Bayes

Phương pháp kết hợp (ensemble method) là cách tiếp cận kết hợp nhiều mô hình "khối xây dựng" đơn giản để thu được một mô hình duy nhất và có khả năng rất mạnh mẽ. Những mô hình khối xây dựng đơn giản này đôi khi được gọi là các mô hình học yếu (weak learners), vì chúng có thể dẫn đến những dự đoán tầm thường khi đứng riêng lẻ.

dàn nhạc

Chúng ta sẽ thảo luận về bagging, rừng ngẫu nhiên, boosting và cây hồi quy cộng tính Bayes. Đây là các phương pháp kết hợp mà khối xây dựng đơn giản là cây hồi quy hoặc cây phân loại.

học
sinh yếu

8.2.1 Đóng bao

Phương pháp bootstrap, được giới thiệu trong Chương 5, là một ý tưởng vô cùng mạnh mẽ. Nó được sử dụng trong nhiều trường hợp khó hoặc thậm chí không thể tính toán trực tiếp độ lệch chuẩn của một đại lượng cần quan tâm. Ở đây, chúng ta thấy rằng phương pháp bootstrap có thể được sử dụng trong một ngữ cảnh hoàn toàn khác, nhằm cải thiện các phương pháp học thống kê như cây quyết định.

Các cây quyết định được thảo luận trong Phần 8.1 có độ biến thiên cao.

Điều này có nghĩa là nếu chúng ta chia dữ liệu huấn luyện thành hai phần ngẫu nhiên và áp dụng cây quyết định cho cả hai phần, kết quả thu được có thể khá khác nhau. Ngược lại, một quy trình có phương sai thấp sẽ cho kết quả tương tự nếu được áp dụng lặp đi lặp lại cho các tập dữ liệu khác nhau; hồi quy tuyến tính có xu hướng có phương sai thấp nếu tỷ lệ n trên p tương đối lớn. Phương pháp tổng hợp Bootstrap, hay bagging, là một quy trình đa năng để giảm phương sai của một phương pháp học thống kê; chúng tôi giới thiệu nó ở đây vì nó đặc biệt hữu ích và thường được sử dụng trong ngữ cảnh của cây quyết định.

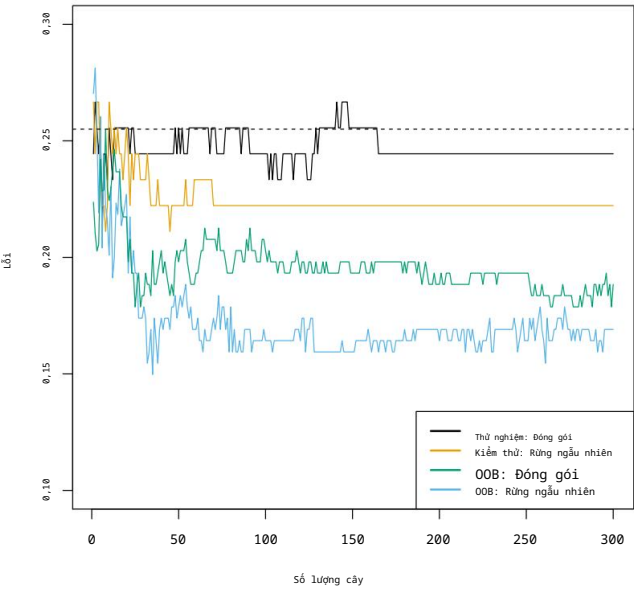
đóng gói

Hãy nhớ rằng, cho một tập hợp n quan sát độc lập $Z_1, \dots, Z_n$, mỗi quan sát có phương sai σ^2 , thì phương sai của giá trị trung bình Z của các quan sát được cho bởi σ^2/n . Nói cách khác, việc lấy trung bình một tập hợp các quan sát sẽ làm giảm phương sai. Do đó, một cách tự nhiên để giảm phương sai và tăng độ chính xác của tập dữ liệu kiểm tra đối với phương pháp học thống kê là lấy nhiều tập dữ liệu huấn luyện từ quần thể, xây dựng một mô hình dự đoán riêng biệt bằng cách sử dụng mỗi tập dữ liệu huấn luyện và tính trung bình các dự đoán thu được. Nói cách khác, chúng ta có thể tính toán $\hat{f}_1(x), \hat{f}_2(x), \dots, \hat{f}_B(x)$ bằng cách sử dụng B tập dữ liệu huấn luyện riêng biệt và tính trung bình chúng để thu được một mô hình học thống kê có phương sai thấp duy nhất, được cho bởi

$$\hat{f}_{\text{avg}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x).$$

Tất nhiên, điều này không thực tế vì chúng ta thường không có quyền truy cập vào nhiều tập dữ liệu huấn luyện. Thay vào đó, chúng ta có thể sử dụng phương pháp bootstrap, bằng cách lấy mẫu lặp đi lặp lại từ tập dữ liệu huấn luyện (duy nhất). Trong phương pháp này, chúng ta tạo ra B tập dữ liệu huấn luyện bootstrap khác nhau. Sau đó, chúng ta huấn luyện phương pháp của mình trên tập dữ liệu huấn luyện bootstrap thứ b để có được $\hat{f}_b(x)$, và cuối cùng tính trung bình tất cả các dự đoán để thu được

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x).$$



HÌNH 8.8. Kết quả của thuật toán Bagging và Random Forest cho dữ liệu **Heart** . Bài kiểm tra Lỗi (màu đen và cam) được hiển thị dưới dạng hàm của B, số lần lấy mẫu bootstrap.

Các tập dữ liệu huấn luyện đã được sử dụng. Thuật toán rừng ngẫu nhiên được áp dụng với $m = \sqrt{p}$. Đường nét đứt cho biết lỗi kiểm thử phát sinh từ một cây phân loại duy nhất. Màu xanh lá cây và Các vạch màu xanh lam thể hiện lỗi OOB, trong trường hợp này – do ngẫu nhiên – lại khá lớn. thấp hơn.

Quá trình này được gọi là đóng gói.

Mặc dù kỹ thuật bagging có thể cải thiện khả năng dự đoán cho nhiều phương pháp hồi quy, Nó đặc biệt hữu ích cho cây quyết định. Để áp dụng bagging vào hồi quy chúng ta chỉ đơn giản là xây dựng B cây hồi quy bằng cách sử dụng huấn luyện bootstrap B. các tập hợp, và tính trung bình các dự đoán thu được. Các cây này được xây dựng sâu, và không được tỉa cành. Do đó, mỗi cây riêng lẻ đều có sự khác biệt lớn, nhưng độ lệch thấp. Việc lấy trung bình các cây B này làm giảm phương sai. Phương pháp bagging đã được sử dụng. đã được chứng minh là mang lại những cải tiến ấn tượng về độ chính xác bằng cách kết hợp Kết hợp hàng trăm hoặc thậm chí hàng nghìn cây vào một quy trình duy nhất.

Cho đến nay, chúng ta đã mô tả quy trình bagging trong hồi quy. Trong ngữ cảnh này, để dự đoán kết quả định lượng Y. Làm thế nào có thể mở rộng phương pháp bagging? đối với một bài toán phân loại trong đó Y là biến định tính? Trong trường hợp đó, có Có một vài cách tiếp cận khả thi, nhưng cách đơn giản nhất như sau. Đối với một bài kiểm tra nhất định Qua quan sát, chúng ta có thể ghi lại lớp được dự đoán bởi mỗi cây B, và Lấy đa số phiếu: dự đoán tổng thể là dự đoán xuất hiện thường xuyên nhất trong số các dự đoán loại B.

số đồng
bỏ phiếu

Hình 8.8 hiển thị kết quả từ việc sử dụng kỹ thuật bagging để phân loại cây trên tập dữ liệu **Heart** . Tỷ lệ lỗi kiểm thử được biểu diễn dưới dạng hàm số của B, tức là số lượng cây được xây dựng. sử dụng các tập dữ liệu huấn luyện bootstrap. Chúng ta thấy rằng lỗi kiểm thử bagging Trong trường hợp này, tỷ lệ lỗi thấp hơn một chút so với tỷ lệ lỗi kiểm tra thu được từ... cây đơn. Số lượng cây B không phải là tham số quan trọng đối với phương pháp bagging; Việc sử dụng giá trị B rất lớn sẽ không dẫn đến hiện tượng quá khớp. Trong thực tế, chúng ta

Hãy sử dụng giá trị B đủ lớn để lỗi ổn định. Sử dụng $B = 100$ là đủ để đạt được hiệu suất tốt trong ví dụ này.

Ước lượng lỗi ngoài mẫu

Hóa ra có một cách rất đơn giản để ước tính sai số kiểm thử của một mô hình bagging mà không cần thực hiện kiểm định chéo hoặc phương pháp tập dữ liệu xác thực. Nhớ lại rằng điểm mấu chốt của bagging là các cây được lặp đi lặp lại để huấn luyện trên các tập con được bootstrap của các quan sát. Có thể chứng minh rằng trung bình, mỗi cây bagging sử dụng khoảng hai phần ba số quan sát.³ Một phần ba số quan sát còn lại không được sử dụng để huấn luyện một cây bagging nhất định được gọi là các quan sát ngoài bag (OOB). Chúng ta có thể dự đoán phản hồi cho quan sát thứ i bằng cách sử dụng mỗi cây mà quan sát đó là OOB. Điều này sẽ tạo ra khoảng $B/3$ dự đoán cho quan sát thứ i . Để có được một dự đoán duy nhất cho quan sát thứ i , chúng ta có thể lấy trung bình các phản hồi được dự đoán này (nếu mục tiêu là hồi quy) hoặc có thể lấy phiếu bầu đa số (nếu mục tiêu là phân loại). Điều này dẫn đến một dự đoán OOB duy nhất cho quan sát thứ i . Theo cách này, có thể thu được dự đoán OOB cho mỗi trong số n quan sát, từ đó có thể tính toán MSE OOB tổng thể (đối với bài toán hồi quy) hoặc lỗi phân loại (đối với bài toán phân loại). Lỗi OOB thu được là một ước lượng hợp lệ về lỗi kiểm định cho mô hình được ghép cặp, vì phản hồi cho mỗi quan sát được dự đoán chỉ bằng cách sử dụng các cây không được huấn luyện bằng quan sát đó. Hình 8.8 hiển thị lỗi OOB trên dữ liệu **Heart**. Có thể thấy rằng với B đủ lớn, lỗi OOB gần như tương đương với lỗi kiểm định chéo loại bỏ một phần tử. Phương pháp OOB để ước tính lỗi kiểm định đặc biệt thuận tiện khi thực hiện ghép cặp trên các tập dữ liệu lớn mà việc kiểm định chéo sẽ tốn nhiều tài nguyên tính toán.

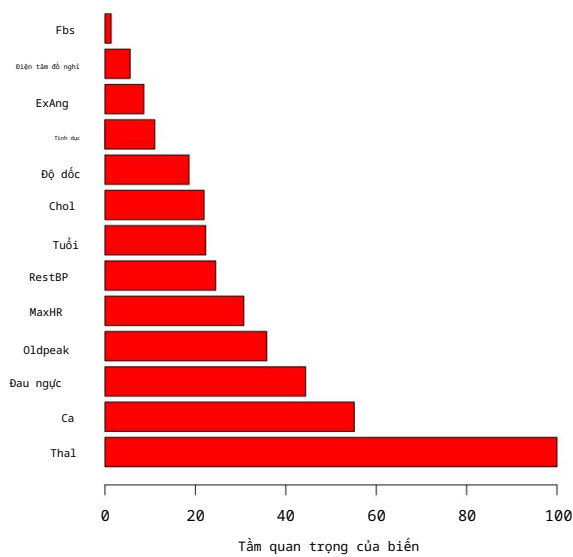
ngoài túi

Các biện pháp đo lường tầm quan trọng của biến

Như chúng ta đã thảo luận, phương pháp bagging thường mang lại độ chính xác cao hơn so với dự đoán bằng một cây quyết định đơn lẻ. Tuy nhiên, thật không may là việc giải thích mô hình thu được có thể khó khăn. Hãy nhớ rằng một trong những ưu điểm của cây quyết định là sơ đồ trực quan và dễ hiểu, chẳng hạn như sơ đồ được hiển thị trong Hình 8.1. Tuy nhiên, khi chúng ta sử dụng bagging cho một số lượng lớn cây, việc biểu diễn quy trình học thống kê thu được bằng một cây duy nhất không còn khả thi nữa, và không còn rõ ràng biến nào là quan trọng nhất đối với quy trình đó. Do đó, bagging cải thiện độ chính xác dự đoán nhưng lại làm giảm khả năng giải thích.

Mặc dù việc diễn giải tập hợp các cây được đóng gói khó hơn nhiều so với một cây đơn lẻ, người ta vẫn có thể thu được bản tóm tắt tổng quan về tầm quan trọng của từng biến dự đoán bằng cách sử dụng RSS (đối với cây hồi quy đóng gói) hoặc chỉ số Gini (đối với cây phân loại đóng gói). Trong trường hợp cây hồi quy đóng gói, chúng ta có thể ghi lại tổng lượng RSS (8.1) giảm do việc phân tách trên một biến dự đoán nhất định, được tính trung bình trên tất cả B cây. Giá trị lớn cho thấy một biến dự đoán quan trọng. Tương tự, trong bối cảnh phân loại đóng gói.

3. Điều này liên quan đến Bài tập 2 của Chương 5.



HÌNH 8.9. Biểu đồ tầm quan trọng của biến đối với dữ liệu *Tim mạch*. Tầm quan trọng của biến được tính toán bằng cách sử dụng mức giảm trung bình của chỉ số Gini và được biểu thị tương đối. tối đa.

cây cối, chúng ta có thể cộng tổng số lượng mà chỉ số Gini (8,6) giảm bằng cách phân tách dựa trên một biến dự đoán nhất định, tính trung bình trên tất cả các cây B.

Hình 8.9 thể hiện đồ thị biểu diễn tầm quan trọng của các biến trong dữ liệu về *Tim*. Chúng ta thấy mức giảm trung bình của chỉ số Gini đối với mỗi biến, so với biến có mức giảm trung bình lớn nhất. Các biến có mức giảm trung bình lớn nhất

Các chỉ số trong chỉ số Gini bao gồm *Thal*, *Ca* và *Đau ngực*.

biến
tầm quan trọng

8.2.2 Rừng ngẫu nhiên

Rừng ngẫu nhiên mang lại sự cải tiến so với cây quyết định bằng phương pháp đóng gói (bagging trees) thông qua một điều chỉnh nhỏ giúp giảm sự tương quan giữa các cây. Cũng như trong phương pháp đóng gói, chúng ta xây dựng một số lượng cây quyết định của cây quyết định trên các mẫu huấn luyện được lấy mẫu ngẫu nhiên. Nhưng khi xây dựng những cây này Trong cây quyết định, mỗi khi xem xét một sự phân nhánh trong cây, một mẫu ngẫu nhiên của m biến dự đoán được chọn làm ứng viên phân tách từ tập hợp đầy đủ gồm p biến dự đoán. Việc chia tách chỉ được phép sử dụng một trong số m biến dự đoán đó. Một mẫu mới của m biến dự đoán được chọn ở mỗi lần chia, và thông thường chúng ta chọn $m \approx \sqrt{p}$ — tức là, số lượng biến dự đoán được xem xét ở mỗi lần chia tách xấp xỉ bằng nhau.

ngẫu nhiên
rừng

Dữ liệu về *tim mạch*).

Nói cách khác, khi xây dựng một rừng ngẫu nhiên, tại mỗi điểm phân nhánh trong cây, Thuật toán thậm chí không được phép xem xét phần lớn các lựa chọn có sẵn. các yếu tố dự báo. Điều này nghe có vẻ điên rồ, nhưng nó có một lý lẽ rất hợp lý. Giả sử Điều đó có nghĩa là có một biến dự báo rất mạnh trong tập dữ liệu, cùng với một số biến dự báo khác có độ mạnh vừa phải. Sau đó, trong tập hợp các biến được đóng gói (bagged), Hầu hết hoặc tất cả các cây quyết định sẽ sử dụng yếu tố dự đoán mạnh mẽ này trong quá trình phân chia nhóm hàng đầu. Do đó, tất cả các cây được đóng gói sẽ trông khá giống nhau.

Do đó, các dự đoán từ các cây quyết định được ghép lại sẽ có mối tương quan cao. Thật không may, việc lấy trung bình nhiều đại lượng có mối tương quan cao không dẫn đến sự giảm phương sai lớn như việc lấy trung bình nhiều đại lượng không tương quan. Cụ thể, điều này có nghĩa là việc ghép cây sẽ không dẫn đến sự giảm phương sai đáng kể so với một cây đơn lẻ trong trường hợp này.

Rừng ngẫu nhiên khắc phục vấn đề này bằng cách buộc mỗi lần phân nhánh chỉ xem xét một tập hợp con các biến dự đoán. Do đó, trung bình $(p - m)/p$ lần phân nhánh thậm chí sẽ không xem xét biến dự đoán mạnh, và vì vậy các biến dự đoán khác sẽ có nhiều cơ hội hơn. Chúng ta có thể coi quá trình này như việc khử tương quan giữa các cây, do đó làm cho giá trị trung bình của các cây kết quả ít biến động hơn và do đó đáng tin cậy hơn.

Sự khác biệt chính giữa bagging và rừng ngẫu nhiên nằm ở việc lựa chọn kích thước tập con dự đoán m . Ví dụ, nếu một rừng ngẫu nhiên được xây dựng bằng cách sử dụng $m = p$, thì điều này đơn giản chỉ là bagging. Trên tập dữ liệu **Heart**, rừng ngẫu nhiên sử dụng $m = \sqrt{p}$ dẫn đến giảm cả lỗi kiểm thử và lỗi OOB so với bagging (Hình 8.8).

Việc sử dụng giá trị m nhỏ khi xây dựng rừng ngẫu nhiên thường hữu ích khi chúng ta có một số lượng lớn các biến dự đoán tương quan. Chúng tôi đã áp dụng rừng ngẫu nhiên cho một tập dữ liệu sinh học đa chiều bao gồm các phép đo biểu hiện của 4.718 gen được đo trên các mẫu mô từ 349 bệnh nhân. Có khoảng 20.000 gen ở người, và mỗi gen có mức độ hoạt động, hay biểu hiện, khác nhau trong các tế bào, mô và điều kiện sinh học cụ thể. Trong tập dữ liệu này, mỗi mẫu bệnh nhân có một nhãn định tính với 15 mức độ khác nhau: bình thường hoặc 1 trong 14 loại ung thư khác nhau. Mục tiêu của chúng tôi là sử dụng rừng ngẫu nhiên để dự đoán loại ung thư dựa trên 500 gen có phương sai lớn nhất trong tập huấn luyện.

Chúng tôi chia ngẫu nhiên các quan sát thành tập huấn luyện và tập kiểm tra, và áp dụng thuật toán rừng ngẫu nhiên cho tập huấn luyện với ba giá trị khác nhau của số lượng biến phân tách m . Kết quả được thể hiện trong Hình 8.10. Tỷ lệ lỗi của một cây đơn là 45,7%, và tỷ lệ cây rỗng là 75,4%.

⁴ Chúng ta thấy rằng việc sử dụng 400 cây là đủ để mang lại hiệu suất tốt, và lựa chọn $m = \sqrt{p}$ đã mang lại một sự cải thiện nhỏ về lỗi kiểm thử so với phương pháp bagging ($m = p$) trong ví dụ này. Cũng như bagging, rừng ngẫu nhiên sẽ không bị quá khớp nếu chúng ta tăng B , vì vậy trên thực tế, chúng ta sử dụng giá trị B đủ lớn để tỷ lệ lỗi ổn định.

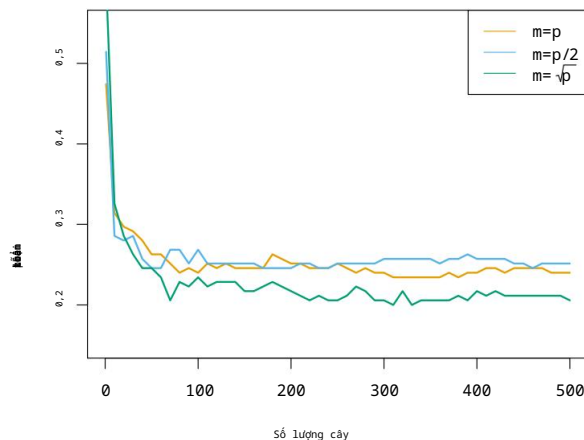
8.2.3 Tăng cường

Tiếp theo, chúng ta sẽ thảo luận về boosting, một phương pháp khác để cải thiện các dự đoán từ cây quyết định. Giống như bagging, boosting là một phương pháp tổng quát có thể được áp dụng cho nhiều phương pháp học thống kê để hồi quy hoặc phân loại. Ở đây, chúng ta sẽ giới hạn thảo luận về boosting trong ngữ cảnh của cây quyết định.

tăng cường

Hãy nhớ rằng, phương pháp bagging bao gồm việc tạo ra nhiều bản sao của tập dữ liệu huấn luyện gốc bằng cách sử dụng bootstrap, huấn luyện một cây quyết định riêng biệt cho mỗi bản sao, và sau đó kết hợp tất cả các cây để tạo ra một cây dự đoán duy nhất.

4. Tỷ lệ không có kết quả là do việc phân loại đơn giản từng quan sát vào lớp chiếm ưu thế nói chung, trong trường hợp này là lớp bình thường.



HÌNH 8.10. Kết quả từ rừng ngẫu nhiên cho biểu hiện gen 15 lớp

Tập dữ liệu với $p = 500$ biến dự đoán. Sai số kiểm định được hiển thị dưới dạng hàm của số lượng cây. Mỗi đường màu tương ứng với một giá trị khác nhau của m , Số lượng biến dự đoán có sẵn để phân tách tại mỗi nút cây bên trong. Ngẫu nhiên Rừng ($m < p$) dẫn đến sự cải thiện nhẹ so với phương pháp đóng bao ($m = p$). Một Cây phân loại có tỷ lệ lỗi là 45,7%.

mô hình tích cực. Đáng chú ý, mỗi cây được xây dựng trên một tập dữ liệu bootstrap, độc lập của những cây khác. Chức năng tăng cường hoạt động tương tự, ngoại trừ việc các cây này... Phát triển tuần tự: mỗi cây được phát triển bằng cách sử dụng thông tin từ các cây trước đó. Cây cối đã phát triển. Việc tăng cường không liên quan đến lấy mẫu bootstrap; thay vào đó, mỗi Cây quyết định được xây dựng dựa trên phiên bản đã được chỉnh sửa của tập dữ liệu gốc.

Trước tiên hãy xem xét thiết lập hồi quy. Giống như bagging, boosting liên quan đến việc kết hợp một số lượng lớn cây quyết định, $\hat{f}_1, \dots, \hat{f}_B$. Boosting được mô tả như sau: trong Thuật toán 8.2.

Ý tưởng đằng sau quy trình này là gì? Không giống như việc áp dụng một cây quyết định lớn duy nhất vào dữ liệu, điều này đồng nghĩa với việc áp dụng dữ liệu một cách cứng nhắc và có thể dẫn đến sai sót. Trong khi đó, phương pháp boosting lại học chậm hơn. Với tình hình hiện tại Với mô hình này, chúng ta áp dụng cây quyết định cho phần dư từ mô hình. Nghĩa là, chúng ta Chúng ta xây dựng một cây quyết định dựa trên phần dư hiện tại, thay vì kết quả phản hồi, như là sự tái-Y. Sau đó, chúng ta thêm cây quyết định mới này vào hàm đã được xây dựng theo thứ tự. để cập nhật phần dư. Mỗi cây này có thể khá nhỏ, chỉ có... một vài nút đầu cuối, được xác định bởi tham số d trong thuật toán. Bằng cách Bằng cách áp dụng các cây quyết định nhỏ vào phần dư, chúng ta dần dần cải thiện $\hat{f}$ ở những khu vực mà nó không hoạt động tốt. Tham số c rút λ làm chậm quá trình. Thậm chí còn tiến xa hơn, cho phép nhiều cây có hình dạng khác nhau tấn công các phần dư. Nói chung, các phương pháp học thống kê học chậm thường có xu hướng... hoạt động tốt. Lưu ý rằng trong boosting, không giống như bagging, cấu trúc của Mỗi cây đều phụ thuộc rất nhiều vào những cây đã mọc trước đó.

Chúng ta vừa mô tả quy trình tăng cường cây hồi quy. Tăng cường Cây phân loại được xây dựng theo cách tương tự nhưng phức tạp hơn một chút, và Các chi tiết cụ thể đã được lược bỏ ở đây.

Thuật toán 8.2 Tăng cường cho cây hồi quy

1. Đặt $\hat{f}(x)=0$ và $r_i = y_i$ cho tất cả i trong tập huấn luyện.

2. Với $b = 1, 2, \dots, B$, hãy lặp lại:

(a) Xây dựng một cây $\hat{f}_b$ với d nhánh ($d + 1$ nút cuối) cho quá trình huấn luyện dữ liệu (X, r) .

(b) Cập nhật $\hat{f}$ bằng cách thêm vào một phiên bản thu nhỏ của cây mới:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}_b(x). \quad (8.10)$$

(c) Cập nhật phần dư,

$$r_i \leftarrow r_i - \lambda \hat{f}_b(x_i). \quad (8.11)$$

3. Xuất mô hình đã được cải tiến.

$$\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}_b(x). \quad (8.12)$$

Chức năng tăng áp có ba thông số điều chỉnh:

1. Số lượng cây B . Không giống như bagging và rừng ngẫu nhiên, boosting

có thể xảy ra hiện tượng quá khớp nếu B quá lớn, mặc dù hiện tượng quá khớp này thường không thường xuyên xảy ra.

Chậm, nếu có thể. Chúng tôi sử dụng phương pháp kiểm định chéo để chọn B .

2. Tham số co rút λ , một số dương nhỏ. Tham số này kiểm soát

Tốc độ học tập của thuật toán boosting. Các giá trị điển hình là 0,01 hoặc 0,001, và

Sự lựa chọn đúng đắn có thể phụ thuộc vào vấn đề. Giá trị λ rất nhỏ có thể đòi hỏi...

Sử dụng giá trị B rất lớn để đạt được hiệu suất tốt.

3. Số lượng d các nhánh rẽ trong mỗi cây, yếu tố này kiểm soát độ phức tạp.

của tập hợp được tăng cường. Thông thường $d = 1$ hoạt động tốt, trong trường hợp đó mỗi

Cây là một gốc cây, bao gồm một nhánh duy nhất. Trong trường hợp này, tập hợp tăng cường

đang phù hợp với một mô hình cộng tính, vì mỗi thuật ngữ chỉ liên quan đến một

biến đơn. Nói chung hơn, d là độ sâu tương tác và kiểm soát thứ tự tương tác của mô hình

tăng cường, vì d phân tách có thể liên quan đến

Tối đa d biến.

gốc cây

sự tương tác

độ sâu

Trong Hình 8.11, chúng tôi đã áp dụng phương pháp tăng cường biểu hiện gen ung thư thuộc 15 lớp.

tập dữ liệu, nhằm mục đích phát triển một bộ phân loại có thể phân biệt được dữ liệu bình thường

lớp từ 14 lớp ung thư. Chúng tôi hiển thị lỗi kiểm tra như một hàm của

tổng số cây và độ sâu tương tác d . Chúng ta thấy rằng đơn giản

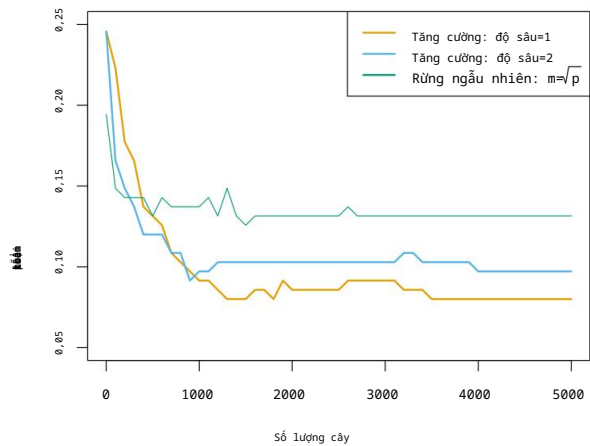
Các gốc cây có độ sâu tương tác bằng một sẽ hoạt động tốt nếu có đủ số lượng chúng.

được bao gồm. Mô hình này hoạt động tốt hơn mô hình độ sâu hai, và cả hai đều hoạt động tốt

hơn rừng ngẫu nhiên. Điều này làm nổi bật một sự khác biệt giữa việc tăng cường

và rừng ngẫu nhiên: trong thuật toán boosting, vì sự phát triển của một cây cụ thể

Tính đến cả những cây khác đã mọc, những cây nhỏ hơn.



HÌNH 8.11. Kết quả từ việc thực hiện thuật toán boosting và random forests trên Bộ dữ liệu biểu hiện gen gồm 15 lớp được sử dụng để dự đoán ung thư so với bình thường. Bài kiểm tra Sai số được hiển thị dưới dạng hàm số của số lượng cây. Đối với hai mô hình tăng cường, $\lambda = 0,01$. Cây có độ sâu 1 hoạt động tốt hơn một chút so với cây có độ sâu 2, và cả hai đều hoạt động tốt hơn. Rừng ngẫu nhiên, mặc dù sai số chuẩn khoảng 0,02, khiến cho không có... Những khác biệt này rất đáng kể. Tỷ lệ lỗi kiểm thử đối với một cây đơn lẻ là 24%.

Thông thường, số lượng cây là đủ. Sử dụng cây nhỏ hơn có thể giúp việc giải thích dễ dàng hơn. Ngoài ra, ví dụ, việc sử dụng gốc cây dẫn đến một mô hình cộng tính.

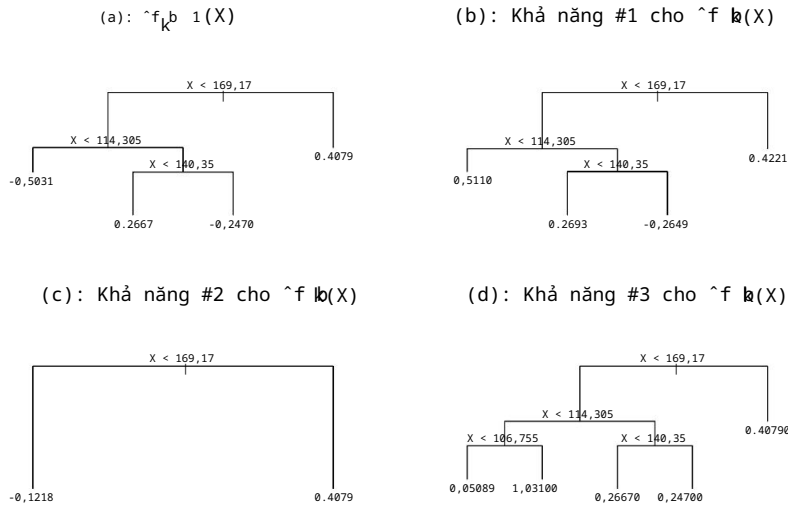
8.2.4 Cây hồi quy cộng tính Bayes

Cuối cùng, chúng ta sẽ thảo luận về cây hồi quy cộng tính Bayes (BART), một phương pháp kết hợp khác sử dụng cây quyết định làm khối xây dựng. Để đơn giản, Chúng tôi giới thiệu BART cho hồi quy (trái ngược với phân loại).
Hãy nhớ rằng thuật toán bagging và random forests đưa ra dự đoán dựa trên giá trị trung bình của các cây hồi quy, mỗi cây được xây dựng bằng cách sử dụng một mẫu dữ liệu ngẫu nhiên. và/hoặc các biến dự đoán. Mỗi cây được xây dựng riêng biệt với các cây khác. Ngược lại, boosting sử dụng tổng trọng số của các cây, mỗi cây được xây dựng bằng cách áp dụng một cây quyết định vào phần dư của phép khớp hiện tại. Do đó, mỗi cây quyết định mới cố gắng nắm bắt tín hiệu chưa được giải thích bởi tập hợp hiện tại của cây. BART có liên quan đến cả hai phương pháp: mỗi cây được xây dựng theo cách thức nào? Tín hiệu bất giữ chưa được mô hình hiện tại tính đến, như trong trường hợp tăng cường tín hiệu. Điểm mới chính trong hệ thống BART là cách thức tạo ra các cây xanh mới.

Bayesian
chất phụ gia
hồi quy
cây cối

Trước khi giới thiệu thuật toán BART, chúng ta cần định nghĩa một số ký hiệu. Chúng ta gọi K là số lượng cây hồi quy và B là số lần lặp. mà thuật toán BART sẽ được chạy. Ký hiệu $\hat{f}_b$ $k(x)$ biểu thị Giá trị dự đoán tại x cho cây hồi quy thứ k được sử dụng trong lần lặp thứ b . Tại Vào cuối mỗi vòng lặp, K cây từ vòng lặp đó sẽ được cộng lại. tức là $\hat{f}_b(x) = \sum_{k=1}^K \hat{f}_b(x)$ với $b = 1, \dots, B$.

Trong lần lặp đầu tiên của thuật toán BART, tất cả các cây đều được khởi tạo thành có một nút gốc duy nhất, với $\hat{f}_1(x) = \frac{1}{nK} \sum_{i=1}^N y_i$, giá trị trung bình của phản hồi



HÌNH 8.12. Sơ đồ cây bị nhiễu từ thuật toán BART. (a):

Cây thứ k ở lần lặp thứ b ($\hat{f}_k^{(b)}(X)$) được hiển thị. Bảng (b)–(d) hiển thị ba trong số nhiều khả năng cho $\hat{f}_k(X)$, cho dạng $\hat{f}_k^{(b)}(X)$. (b): Một khả năng là $\hat{f}_k^{(b)}(X)$ có cấu trúc giống như $\hat{f}_k^{(b)}(X)$, nhưng với các giá trị khác nhau dự đoán tại các nút cuối. (c): Một khả năng khác là $\hat{f}_k^{(b)}(X)$ kết quả $k(X)$ từ việc cắt tỉa $\hat{f}_k^{(b)}(X)$. (d): Ngoài ra, $\hat{f}_k^{(b)}(X)$ có thể có nhiều nút cuối hơn $\hat{f}_k^{(b)}(X)$.

giá trị chia cho tổng số cây. Do đó, $\hat{f}_1(X) = \frac{1}{N} \sum_{i=1}^N y_i$.

Trong các lần lặp tiếp theo, BART cập nhật từng cây trong số K cây, mỗi lần một cây. thời gian. Trong lần lặp thứ b , để cập nhật cây thứ k , chúng ta trừ đi từ mỗi giá trị phản hồi là các dự đoán từ tất cả các cây ngoại trừ cây thứ k , để thu được phần dư

$$r_i = y_i - \sum_{k < k} \hat{f}_k^{(b)}(x_i) - \sum_{k > k} \hat{f}_k^{(b)}(x_i)$$

đối với quan sát thứ i , $i = 1, \dots, n$. Thay vì xây dựng một cây mới cho điều này phần dư một phần, BART chọn ngẫu nhiên một nhiễu loạn cho cây từ lần lặp trước ($\hat{f}_k^{(b)}(X)$) từ một tập hợp các nhiễu loạn có thể xảy ra, ưu tiên những yếu tố giúp cải thiện độ phù hợp với phần dư riêng phần. Có hai thành phần đối với sự nhiễu loạn này:

1. Chúng ta có thể thay đổi cấu trúc của cây bằng cách thêm hoặc tỉa cành.
2. Chúng ta có thể thay đổi dự đoán ở mỗi nút cuối cùng của cây.

Hình 8.12 minh họa các ví dụ về những tác động có thể xảy ra đối với một cái cây.

Kết quả đầu ra của BART là một tập hợp các mô hình dự đoán.

$$\hat{f}_b(X) = \sum_{k=1}^K \hat{f}_k(X), \text{ với } b = 1, 2, \dots, B.$$

Thuật toán 8.3 Cây hồi quy cộng tính Bayes

1. Cho $\hat{f}_1(x) = \hat{f}_2(x) = \dots = \hat{f}_K(x) = \frac{1}{nK} \sum_{t=1}^N y_i$.
2. Tính $\hat{f}_1(x) = \sum_{k=1}^K \hat{f}_k(x) = \frac{1}{N} \sum_{t=1}^N y_i$.
3. Với $b = 2, \dots, B$:

(a) Đối với $k = 1, 2, \dots, K$:

i. Với $i = 1, \dots, n$, hãy tính phần dư riêng hiện tại.

$$r_i = y_i - \sum_{k < k} \hat{f}_k(x_i) - \sum_{k > k} \hat{f}_k^{b-1}(x_i).$$

ii. Lắp một cây mới, $\hat{f}_k^b(x)$, đến r_i , bằng cách làm nhiều ngẫu nhiên cây thứ k từ lần lặp trước, $\hat{f}_k^{b-1}(x)$. Sự nhiễu loạn Những yếu tố giúp cải thiện độ vùa vắn sẽ được ưu tiên.

(b) Tính $\hat{f}_b(x) = \sum_{k=1}^K \hat{f}_k^b(x)$.
4. Tính giá trị trung bình sau khi lấy L mẫu khởi động (burn-in samples).

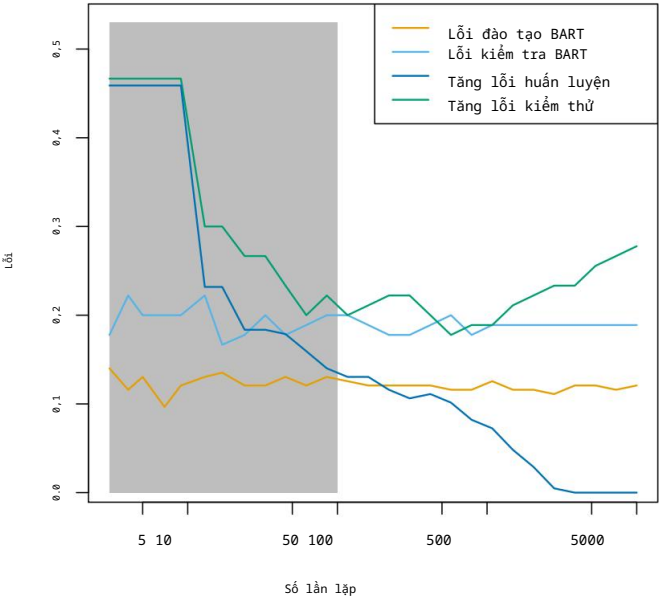
$$\hat{f}(x) = \frac{1}{B} \sum_{b=L+1}^B \hat{f}_b(x).$$

Chúng ta thường loại bỏ một vài mô hình dự đoán đầu tiên, vì Các mô hình thu được trong các lần lặp đầu tiên – được gọi là giai đoạn khởi động – thường không cho kết quả tốt. Ta có thể ký hiệu L là số lần lặp khởi động; ví dụ, ta có thể lấy $L = 200$. Sau đó, để Để có được một dự đoán duy nhất, chúng ta chỉ cần lấy giá trị trung bình sau giai đoạn khởi động (burn-in). lần lặp, $\hat{f}(x) = \frac{1}{B} \sum_{b=L+1}^B \hat{f}_b(x)$. Tuy nhiên, cũng có thể tính toán các đại lượng khác ngoài giá trị trung bình: ví dụ, các phân vị của $\hat{f}_{L+1}(x), \dots, \hat{f}_B(x)$ cung cấp thước đo độ không chắc chắn trong dự đoán cuối cùng. Quy trình BART tổng thể được tóm tắt trong Thuật toán 8.3.

Một yếu tố quan trọng của phương pháp BART là ở Bước 3(a)ii., chúng ta không phù hợp một cây mới cho phần dư hiện tại: thay vào đó, chúng ta cố gắng cải thiện sự phù hợp. đến phần dư hiện tại bằng cách sửa đổi nhẹ cây thu được trong lần lặp trước (xem Hình 8.12). Nói một cách đại khái, điều này giúp ngăn ngừa Hiện tượng quá khớp (overfitting) xảy ra vì nó giới hạn mức độ "cứng" mà chúng ta điều chỉnh dữ liệu trong mỗi lần lặp. Hơn nữa, các cây riêng lẻ thường khá nhỏ. Chúng tôi giới hạn Kích thước cây quyết định nhằm tránh tình trạng quá khớp dữ liệu, điều này sẽ dễ xảy ra hơn. Điều đó sẽ xảy ra nếu chúng ta trồng những cây rất lớn.

Hình 8.13 thể hiện kết quả áp dụng BART vào dữ liệu **Tim**, sử dụng $K = 200$ cây, khi số lần lặp tăng lên 10.000. Trong quá trình này Trong những lần lặp ban đầu, lỗi kiểm thử và lỗi huấn luyện có sự biến động khá lớn. Sau đó Trong giai đoạn khởi động ban đầu này, tỷ lệ lỗi sẽ ổn định. Chúng tôi lưu ý rằng có Sự khác biệt giữa lỗi huấn luyện và lỗi kiểm tra là rất nhỏ. Điều này cho thấy quá trình nhiễu loạn cây quyết định phần lớn tránh được hiện tượng quá khớp dữ liệu.

sự chảy



HÌNH 8.13. Kết quả BART và boosting cho dữ liệu *Tim* . Cả hai quá trình huấn luyện và các lỗi kiểm tra được hiển thị. Sau giai đoạn chạy thử 100 lần (được hiển thị trong (màu xám), tỷ lệ lỗi của BART ổn định. Boosting bắt đầu bị quá khớp sau một thời gian. vài trăm lần lặp.

Các lỗi trong quá trình huấn luyện và kiểm thử đối với thuật toán boosting cũng được hiển thị trong Hình 8.13. Chúng ta thấy rằng sai số kiểm thử của phương pháp boosting tiến gần đến sai số của phương pháp BART, nhưng sau đó Giá trị này bắt đầu tăng lên khi số lần lặp tăng. Hơn nữa, Lỗi huấn luyện đối với thuật toán boosting giảm khi số lần lặp tăng lên. Điều này cho thấy thuật toán boosting đã làm cho dữ liệu bị quá khớp.

Mặc dù các chi tiết nằm ngoài phạm vi cuốn sách này, nhưng hóa ra... Phương pháp BART có thể được xem như một cách tiếp cận Bayes để điều chỉnh một tập hợp các cây: mỗi lần chúng ta ngẫu nhiên làm nhiều một cây để phù hợp với Trên thực tế, chúng ta đang vẽ một cây mới từ phân bố hậu nghiệm dựa trên phần dư . (Dĩ nhiên, mỗi liên hệ Bayes này chính là động lực cho tên gọi BART.) Hơn nữa, Thuật toán 8.3 có thể được xem như một thuật toán Monte Carlo chuỗi Markov để phù hợp với mô hình BART.

Markov
chuỗi Monte
Carlo

Khi áp dụng BART, chúng ta phải chọn số lượng cây K , tức là số lượng số lần lặp B và số lần lặp khởi động L . Chúng ta thường chọn Giá trị lớn cho B và K , và giá trị vừa phải cho L : ví dụ, $K = 200$, $B = 1.000$ và $L = 100$ là một lựa chọn hợp lý. BART đã được chứng minh là có hiệu quả. có hiệu năng ấn tượng ngay từ khi xuất xưởng – nghĩa là, nó hoạt động tốt. Với sự tinh chỉnh tối thiểu.

8.2.5 Tóm tắt các phương pháp tập hợp cây quyết định

Cây quyết định là một lựa chọn hấp dẫn cho mô hình học yếu trong phương pháp kết hợp. vì một số lý do, bao gồm tính linh hoạt và khả năng xử lý của chúng.

Các yếu tố dự báo thuộc nhiều loại khác nhau (tức là cả định tính lẫn định lượng). Chúng ta đã xem xét bốn phương pháp để xây dựng một tập hợp các cây quyết định: bagging, random forests, boosting và BART.

- Trong phương pháp bagging, các cây được xây dựng độc lập trên các mẫu ngẫu nhiên của các quan sát. Do đó, các cây có xu hướng khá giống nhau. Vì vậy, bagging có thể bị mắc kẹt trong các điểm tối ưu cục bộ và không thể khám phá triệt để không gian mô hình.
- Trong rừng ngẫu nhiên, các cây được xây dựng độc lập trên các mẫu ngẫu nhiên của các quan sát. Tuy nhiên, mỗi lần phân tách trên mỗi cây được thực hiện bằng cách sử dụng một tập con ngẫu nhiên của các đặc trưng, do đó làm giảm sự tương quan giữa các cây và dẫn đến việc khám phá không gian mô hình kỹ lưỡng hơn so với phương pháp bagging.
- Trong phương pháp boosting, chúng ta chỉ sử dụng dữ liệu gốc và không lấy mẫu ngẫu nhiên. Các cây được xây dựng liên tiếp bằng cách sử dụng phương pháp học "chậm": mỗi cây mới được điều chỉnh sao cho phù hợp với tín hiệu còn lại từ các cây trước đó, và được thu nhỏ lại trước khi sử dụng.
- Trong BART, chúng tôi một lần nữa chỉ sử dụng dữ liệu gốc và phát triển các cây một cách tuần tự. Tuy nhiên, mỗi cây đều được điều chỉnh để tránh các cực tiểu cục bộ và đạt được sự khám phá toàn diện hơn về không gian mô hình.

8.3 Bài thực hành: Phương pháp dựa trên cây

Chúng tôi nhập một số thư viện thông dụng của mình ở cấp độ cao nhất này.

```
Trong [1]: import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots
from statsmodels.datasets import get_rdataset
import sklearn.model_selection as skm
from ISLP import load_data, confusion_table
import ISLP.models
import ModelSpec as MS
```

Chúng tôi cũng thu thập các vật tư nhập khẩu mới cần thiết cho phòng thí nghiệm này.

```
Trong [2]: from sklearn.tree import (DecisionTreeClassifier as DTC,
                                   DecisionTreeRegressor as DTR,
                                   plot_tree,
                                   export_text)
from sklearn.metrics import (accuracy_score,
                              log_loss)
from sklearn.ensemble import \
    (RandomForestRegressor as RF,
     GradientBoostingRegressor as GBR)
from ISLP.bart import BART
```

8.3.1 Xây dựng cây phân loại

Trước tiên, chúng ta sử dụng cây phân loại để phân tích tập dữ liệu `Carseats`. Trong tập dữ liệu này, `Sales` là một biến liên tục, vì vậy chúng ta bắt đầu bằng cách mã hóa lại nó thành một biến nhị phân. Chúng ta sử dụng hàm `where()` để tạo một biến có tên là `High`, biến này nhận giá trị là `Yes` nếu biến `Sales` vượt quá 8 và nhận giá trị là `No` trong trường hợp ngược lại.

Ở đâu()

```
Trong [3]: Carseats = load_data('Carseats')
Cao = np.where(Carseats.Sales > 8, "Có", "Không")
```

Giờ đây, chúng ta sử dụng `DecisionTreeClassifier()` để xây dựng cây phân loại nhằm dự đoán "Cao" bằng cách sử dụng tất cả các biến trừ "Doanh số". Để làm được điều đó, chúng ta phải tạo ma trận mô hình giống như khi xây dựng mô hình hồi quy.

Cây quyết định
Bộ phân loại()

```
Trong [4]: model = MS(Carseats.columns.drop('Sales'), intercept=False)
D = model.fit_transform(Carseats) feature_names
= list(D.columns)
X = np.asarray(D)
```

Chúng ta đã chuyển đổi `D` từ một data frame thành một mảng `X`, điều này cần thiết cho một số phân tích bên dưới. Chúng ta cũng cần `feature_names` để chú thích các biểu đồ sau này.

Cần có một số tùy chọn để chỉ định bộ phân loại, chẳng hạn như `max_depth` (độ sâu của cây), `min_samples_split` (số lượng quan sát tối thiểu trong một nút để đủ điều kiện phân tách) và `criterion` (sử dụng Gini hay entropy chéo làm tiêu chí phân tách). Chúng tôi cũng đặt `random_state` để đảm bảo tính khả thi; các trường hợp trùng lặp trong tiêu chí phân tách sẽ được giải quyết ngẫu nhiên.

```
Trong [5]: clf = DTC(tiêu chí='entropy',
                    max_depth=3,
                    random_state=0)

clf.fit(X, High)
```

```
Out[5]: DecisionTreeClassifier(criterion='entropy', max_depth=3)
```

Trong phần thảo luận về các đặc điểm định tính ở Mục 3.3, chúng ta đã lưu ý rằng đối với mô hình hồi quy tuyến tính, đặc điểm đó có thể được biểu diễn bằng cách bao gồm một ma trận các biến giả (mã hóa one-hot) trong ma trận mô hình, sử dụng ký hiệu công thức của `statsmodels`. Như đã đề cập trong Mục 8.1, có một cách tự nhiên hơn để xử lý các đặc điểm định tính khi xây dựng cây quyết định, không yêu cầu các biến giả như vậy; mỗi lần phân tách tương đương với việc phân chia các mức thành hai nhóm. Tuy nhiên, việc triển khai cây quyết định của `sklearn` không tận dụng cách tiếp cận này; thay vào đó, nó chỉ đơn giản coi các mức được mã hóa one-hot là các biến riêng biệt.

```
Trong [6]: accuracy_score(High, clf.predict(X))
```

```
Out[6]: 0.7275
```

Với các tham số mặc định, tỷ lệ lỗi huấn luyện là 21%. Đối với cây phân loại, ta có thể truy cập giá trị độ lệch bằng cách sử dụng `log_loss()`.

log_loss()

$$-2 \sum_{k=1}^m \log \hat{p}_{mk},$$

trong đó m_k là số lượng quan sát tại nút cuối thứ m .
thuộc lớp thứ k .

```
Trong [7]: resid_dev = np.sum(log_loss(High, clf.predict_proba(X)))
          resid_dev
```

Out[7]: 0.4711

Điều này có liên quan mật thiết đến entropy, được định nghĩa trong (8.7). Một độ lệch nhỏ
Biểu thị một cây quyết định phù hợp tốt với dữ liệu (huấn luyện).
Một trong những đặc tính hấp dẫn nhất của cây là chúng có thể được hiển thị dưới
dạng đồ thị. Ở đây, chúng ta sử dụng hàm `plot()` để hiển thị cấu trúc cây.
(không được hiển thị ở đây).

```
Trong [8]: ax = subplots(figsize=(12,12))[1]
          plot_tree(clf,
                    tên_tính_dụng=tên_tính_dụng,
                    ax=ax);
```

Chỉ số quan trọng nhất về doanh số bán hàng dường như là `ShelveLoc`.
Chúng ta có thể xem biểu diễn dạng văn bản của cây bằng cách sử dụng hàm
`export_text()`, hàm này hiển thị tiêu chí phân nhánh (ví dụ: `Giá <= 92.5`) cho mỗi nhánh. ^{xuất_văn bản()} Đối với lá
Các nút này hiển thị dự đoán tổng thể (`Có` hoặc `Không`). Chúng ta cũng có thể thấy con số.
các quan sát trong lá đó nhận giá trị `Có` và `Không` bằng cách chỉ định
`show_weights=True`.

```
Trong [9]: print(export_text(clf,
                             tên_tính_dụng=tên_tính_dụng,
                             show_weights=True))
```

```
Out[9]: |--- ShelveLoc[Good] <= 0.50
        |--- Giá <= 92.50
        | |--- Thu nhập <= 57.00
        | | |--- trọng lượng: [7.00, 3.00] hạng: Không
        | |--- Thu nhập > 57.00
        | | |--- trọng lượng: [7.00, 29.00] hạng: Có
        |--- Giá > 92.50
        | | | | |--- Quảng cáo <= 13.50
        | | | |--- trọng lượng: [183.00, 41.00] hạng: Không
        | | | |--- Quảng cáo > 13.50
        | | | |--- trọng lượng: [20.00, 25.00] hạng: Có
        |--- ShelveLoc[Good] > 0.50
        | |--- Giá <= 135.00
        | | |--- Hoa Kỳ[Có] <= 0,50
        | | | |--- trọng lượng: [6.00, 11.00] hạng: Có
        | | | |--- Hoa Kỳ[Có] > 0,50
        | | | |--- trọng lượng: [2.00, 49.00] hạng: Có
        | |--- Giá > 135.00
        |--- Thu nhập <= 46,00
        | |--- trọng lượng: [6.00, 0.00] hạng: Không
        | |--- Thu nhập > 46,00
        | | |--- trọng lượng: [5.00, 6.00] hạng: Có
```

Để đánh giá chính xác hiệu suất của cây phân loại trên tập dữ liệu này, chúng ta phải ước tính lỗi kiểm thử thay vì chỉ tính toán lỗi huấn luyện. Chúng ta chia các quan sát thành tập huấn luyện và tập kiểm thử, xây dựng cây bằng cách sử dụng tập huấn luyện và đánh giá hiệu suất của nó trên dữ liệu kiểm thử. Mô hình này tương tự như trong Chương 6, với các mô hình tuyến tính được thay thế ở đây bằng cây quyết định – mà để xác thực gần như giống hệt nhau. Cách tiếp cận này dẫn đến dự đoán chính xác cho 68,5% vị trí trong tập dữ liệu kiểm thử.

```
Trong [10]: validation = skm.ShuffleSplit(n_splits=1, test_size=200,
                                         random_state=0)

          results = skm.cross_validate(clf,
                                         D,
                                         Cao,
                                         cv=xác thực)

          kết quả['test_score']
```

Out[10]: array([0.685])

Tiếp theo, chúng ta xem xét liệu việc tĩa cây có thể dẫn đến cải thiện hiệu suất phân loại hay không. Trước tiên, chúng ta chia dữ liệu thành tập huấn luyện và tập kiểm tra. Chúng tôi sẽ sử dụng phương pháp kiểm định chéo để tĩa cây quyết định trên tập dữ liệu huấn luyện, sau đó đánh giá hiệu năng của cây đã được tĩa trên tập dữ liệu kiểm tra.

```
Trong [11]: (X_train,
           X_test,
           High_train,
           High_test) = skm.train_test_split(X, High,
                                              test_size=0.5,
                                              random_state=0)
```

Đầu tiên, chúng ta huấn luyện lại toàn bộ cây quyết định trên tập dữ liệu huấn luyện; ở đây chúng ta không đặt tham số `max_depth` , vì chúng ta sẽ học được tham số đó thông qua phương pháp kiểm định chéo.

```
Trong [12]: clf = DTC(criterion='entropy', random_state=0)
          clf.fit(X_train, High_train)
          accuracy_score(High_test, clf.predict(X_test))
```

Out[12]: 0.735

Tiếp theo, chúng ta sử dụng phương thức `cost_complexity_pruning_path()` của `clf` để trích xuất các giá trị độ phức tạp chi phí.

chi
phí_ độ phức
tạp_ cắt
tĩa_ đường dẫn()

```
Trong [13]: ccp_path = clf.cost_complexity_pruning_path(X_train, High_train) kfold = skm.KFold(10,
                                                                                             random_state=1,
                                                                                             shuffle=True)
```

Điều này tạo ra một tập hợp các tập chất và giá trị α , từ đó chúng ta có thể trích xuất giá trị tối ưu bằng phương pháp kiểm định chéo.

```
Trong [14]: Grid = skm.GridSearchCV(clf,
                                   {'ccp_alpha': ccp_path.ccp_alphas}, refit=True,
```